Simulated Self-Driving Car: Digital Twin, Synthetic Edge Cases and a Verified, Fault-Tolerant Planning Stack

Vishwas N.
M.Tech, Digital Electronics and Communication, M.S. Ramaiah Institute of Technology (MSRIT)

November 4, 2026

Abstract

A pedestrian-crossing world with a lidar-equipped car is used to build and test (i) a NeRF-style spacetime digital twin for "what-if" replay, (ii) a guided diffusion model that synthesises rare hazards, (iii) a behaviour tree with exhaustive and bounded model-checking, (iv) a space-time lattice planner with trajectory optimisation and PD tracking, (v) fault injection with a health monitor, and (vi) C export of the learned policy. Everything is pure NumPy with hand-derived gradients. Measured results: the twin agrees with the real simulator on collision outcome in 96.2% of 52 what-if cases; guided edge-case data lowers the learned policy's hazard-set collision rate from $19.7 \pm 1.6\%$ to $8.2 \pm 0.6\%$; the behaviour tree passes all 6 properties in all 32 abstract states and has 0 violations in 18000 bounded-model-check scenarios; with lidar perception it collides in 3.2% of the 800 hazard scenarios. CARLA, public datasets and on-target embedded timing are *not* yet run (Section 8).

Contents

1 Objectives and status **2**

2 System architecture **3**

3 Methodology **3**

3.1 World, sensor, scenarios . . . 3
3.2 Learned policy (DAgger) . . . 4
3.3 NeRF-style spacetime twin . . . 4
3.4 Diffusion edge cases with guidance . . . 4
3.5 Behaviour tree and verification . . . 4
3.6 Lattice planner, optimisation, tracking . . . 4
3.7 Fault injection and monitor . . . 5
3.8 Embedded export . . . 5

4 Implementation **5**

5 Completed experiments **6**

5.1 E1: Learned policy and diffusion-generated edge cases . . . 6
5.2 E2: NeRF-style digital twin . . . 7
5.3 E3: Behaviour-tree verification . . . 8

5.4 E4: Planner, optimisation and tracking . . . 9
5.5 E5: Fault injection . . . 10
5.6 E6: Embedded deployment (host CPU) . . . 11
5.7 E7: Dataset adapter (software test only) . . . 11

6 Safety argument **11**

7 Real-world applications **12**

8 Experiments planned for this week **12**

9 Future experiments, improvements and observations **13**

A Extending this report **13**

B Source code **14**
B.1 `paths.py` . . . 14
B.2 `nn.py` . . . 14
B.3 `sim.py` . . . 15
B.4 `driving.py` . . . 18
B.5 `scenario_gen.py` . . . 19
B.6 `nerf2d.py` . . . 21
B.7 `stack.py` . . . 24
B.8 `datasets.py` . . . 28
B.9 `carla_bt.py` . . . 29
B.10 `embedded.py` . . . 30
B.11 `run_experiments.py` . . . 31
B.12 `run_twin.py` . . . 32
B.13 `run_stack.py` . . . 34
B.14 `make_figures.py` . . . 36
B.15 `make_figures_stack.py` . . . 40
B.16 `tests.py` . . . 41
B.17 `Makefile` . . . 42

1 Objectives and status

1. Replay logged drives as an editable digital twin and test "what-if" variants of a dangerous event.
2. Generate rare, avoidable hazards with a diffusion model and measure whether they make a learned policy safer.
3. Build a rule-based behaviour tree (BT) that is formally checked, then compare it with a learned policy and a planner.
4. Add trajectory optimisation, tracking control, fault injection and a sensor/brake health monitor.
5. Export the policy to dependency-free C and measure size, latency and int8 accuracy.
6. Document a safety argument whose claims point to measured evidence.

Item	Status	Evidence
Simulator, expert, DAgger policy	**Completed**	Sec. 5.1
NeRF-style twin, what-if replay	**Completed**	Sec. 5.2
Diffusion edge cases (5 seeds)	**Completed**	Sec. 5.1
BT verification (exhaustive + bounded)	**Completed**	Sec. 5.3
Lattice planner, optimisation, PD tracking	**Completed**	Sec. 5.4
Fault injection + monitor (5 faults)	**Completed**	Sec. 5.5
C export, host latency, int8 accuracy	**Completed** (host CPU only)	Sec. 5.6
KITTI-format dataset adapter	**Completed** (unit test on synthetic file only)	Sec. 5.7
Behaviour tree in CARLA	**Written, not executed**	`carla_bt.py`; Sec. 8
Real datasets (KITTI, nuScenes, Waymo)	**Planned**	Sec. 8
On-target microcontroller timing	**Planned**	Sec. 8

Table 1: Status of every item. Only "Completed" items have results in this report.

2 System architecture

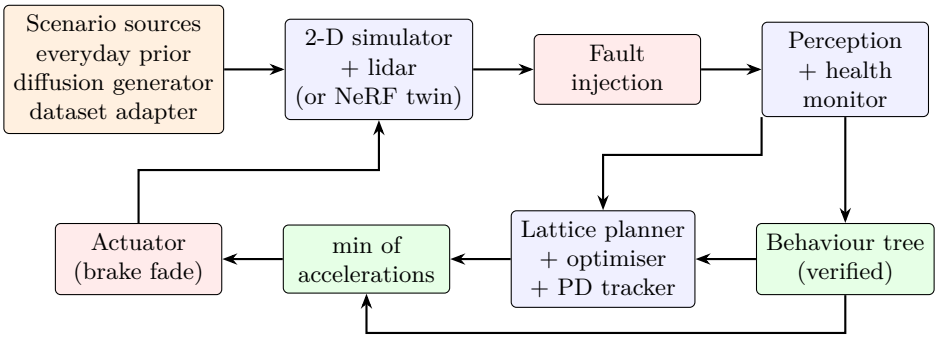


Figure 1: Closed loop. The BT acts as a safety supervisor: the final command is the smaller of planner and BT accelerations.

Folder structure.

```
sdc_project/
  main.tex Makefile requirements.txt
  sections/ 01_overview ... 07_appendix.tex (one file per topic)
  code/ nn sim driving scenario_gen nerf2d stack datasets carla_bt embedded
            run_experiments run_twin run_stack make_figures make_figures_stack tests paths .py
  results/ *.json *.npz macros.tex macros_stack.tex (generated)
  figures/ *.pdf *.png (generated)
  logs/ build/ (generated)
```

3 Methodology

3.1 World, sensor, scenarios

The ego car (4.5 m × 1.9 m) drives along $+x$ with longitudinal control only ($a \in [-8, 3]\,\mathrm{m/s^2}$, $\Delta t = 0.1\,\mathrm{s}$, 18 s). A pedestrian waits at $y = +5.5\,\mathrm{m}$, possibly behind a parked car, and crosses at $x =$ `dist`. A 64-ray lidar (120°, 40 m) adds Gaussian range noise and ray dropout. A scenario is $z = (v_0, \texttt{dist}, s, \tau, o, r, p_{\mathrm{drop}}, \sigma)$ with τ the pedestrian's arrival offset at the lane centre relative to the ego's constant-speed arrival ($\tau \approx 0$: collision course). Collision: gap =

$d(\text{ego rectangle}, \text{pedestrian centre}) - r \le 0$. Everyday data make hazards rare (15% of τ within $\pm 3\,\text{s}$); the hazard test set keeps uniformly drawn scenarios that collide if the ego does not react.

3.2 Learned policy (DAgger)

MLP $65 \to 64 \to 64 \to 1$ on min-pooled lidar, its temporal difference and speed; trained on expert labels with DAgger (rounds with expert mixing $\beta = 1, 0.5, 0$), MSE loss, Adam.

3.3 NeRF-style spacetime twin

A ray with samples r_i, $\delta_i = r_{i+1} - r_i$ renders the expected hit range

$$\hat{D} = r_1 + \sum_i E_i \delta_i, \quad E_i = \exp\Big(-\sum_{j \le i} \sigma_j \delta_j\Big), \qquad \frac{\partial \hat{D}}{\partial \sigma_k} = -\delta_k \sum_{i \ge k} E_i \delta_i. \tag{1}$$

Density is $\sigma(x, y, t) = \sigma_s(x, y) + \sigma_d(x, y, t)$, each $= 20\,\text{softplus}(\text{MLP}(\text{interpolated feature grid}))$. Querying σ_d at $t + \Delta$ shifts the pedestrian in time; the policy is then driven closed-loop on rendered lidar.

3.4 Diffusion edge cases with guidance

A DDPM (T=200) on normalised z minimises $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$. A surrogate $g_\phi(x_t, t)$ predicts the baseline policy's normalised minimum gap; sampling uses

$$\tilde{\epsilon} = \epsilon_\theta + \lambda \sqrt{1 - \bar{\alpha}_t}\, \nabla_{x_t} g_\phi(x_t, t), \qquad \lambda = 8. \tag{2}$$

3.5 Behaviour tree and verification

Priority selector: (1) emergency brake $a = -a_b$; (2) brake to stop line $a = \min(a_{\text{cruise}}, -\text{clip}(1.3\, a_{\text{req}} + 0.3, 0, a_b))$; (3) yield $a = \min(a_{\text{cruise}}, 0)$; (4) cruise to 12 m/s (8 near an occluder, 4 if the sensor is unhealthy, 6 if brakes are degraded). Here $a_{\text{req}} = v^2/(2d)$, d is the distance to a stop line before the pedestrian, and $a_b \in \{8, 4\}$ the braking capability. A conflict is predicted from the pedestrian's entry/exit times with margins $(0.3, 0.8)\,\text{s}$; a pedestrian waiting at the curb is assumed to start at 3.5 m/s. *Verification.* (a) Exhaustive: all 32 abstract states (hazard level × occluder × sensor flag × a_b), running the real action code, against 6 properties (Table 5). (b) Bounded model check: simulate the tree with true pedestrian state on a grid of $v_0 \in \{6..14\}$, `dist`, $s \in \{0.8..3.5\}$, $\tau \in [-6, 6]$ (step 0.5), occluder, $r \in \{0.25, 0.6, 1.0\}$: 18000 scenarios. This is a finite grid, not a proof for all continuous inputs.

3.6 Lattice planner, optimisation, tracking

Layered dynamic programming over (t, s, v) nodes (0.5 s layers, 6 s horizon, $a \in \{-6, -3, -1.5, 0, 1, 2\}$) with edges rejected if the predicted pedestrian comes within 1 m; cost per edge $\Delta t\big[(v_d - \bar{v})^2 + 0.15 a^2\big]$. The lattice path s^{lat} is refined by

$$\min_s \|s - s^{\text{lat}}\|^2 + \lambda_j \|\dddot{s}\|^2 \;\; \text{s.t. } |s_k - s_k^{\text{lat}}| \le 0.4\,\text{m}, \;\; \lambda_j = 2 \cdot 10^{-5}, \tag{3}$$

solved as a linear system then clipped to the tube. Tracking: $a = a_{\text{ref}} + k_p(s_{\text{ref}} - s) + k_d(v_{\text{ref}} - v)$ (k_p=2, k_d=2.5) through a first-order actuator lag (0.25 s). Replanning at 2 Hz.

3.7 Fault injection and monitor

Faults start 1 s before the pedestrian begins to cross: *dropout* (all rays lost, 2 s), *frozen frame* (2 s), *range bias* ($\times 1.25$, persistent), *latency* (0.3 s, persistent), *brake fade* (decel $\geq -4\,\mathrm{m/s^2}$, persistent). Monitor: sensor flagged unhealthy for 2 s if $> 30\%$ of wall rays read beyond the known wall by 1.5 m or the frame is bit-identical to the previous one; brakes flagged degraded if achieved deceleration $< 70\%$ of a hard command.

3.8 Embedded export

Per-tensor symmetric int8 weights $q = \mathrm{round}(W/s)$, $s = \max|W|/127$, float activations; generated C has no libraries.

4 Implementation

All code is in `code/` (NumPy and Matplotlib only; Python 3.10+). Complete listings are in Appendix B.

File	Role
`nn.py`	MLP with manual back-propagation, Adam
`sim.py`	batched simulator, lidar, scenario samplers, open-loop hazard test
`driving.py`	privileged expert, DAgger, evaluation
`nerf2d.py`	spacetime neural field, volume rendering, twin simulator
`scenario_gen.py`	DDPM over scenarios, surrogate, guidance
`stack.py`	behaviour tree, perception, monitor, fault injection, lattice planner, optimiser, tracker
`embedded.py`	C export (float32/int8), host benchmark, int8 closed loop
`datasets.py`	KITTI tracking labels → scenario vectors
`carla_bt.py`	same BT driving a CARLA vehicle (needs CARLA server)
`run_experiments.py`, `run_twin.py`, `run_stack.py`	experiments E1, E2, E3–E5
`make_figures*.py`	figures and LaTeX number macros
`tests.py`, `paths.py`	smoke tests, output folders

Table 2: Code base.

Run.

```
pip install -r requirements.txt
make test # smoke tests
make results # E1-E6, about 15 min on 2 CPU cores, writes results/ and logs/
make figures # writes figures/ and results/macros*.tex
make pdf # compiles this report (or upload the folder to Overleaf)
```

Design points. Every scenario batch is vectorised, so thousands of episodes run at once; the BT is vectorised (each node returns a status array); all experiments are seeded. Perception removes the known parked car using the static map, clusters the remaining returns, and estimates pedestrian velocity by a least-squares line over the last 1 s; with < 4 valid samples it assumes a crossing at 1.5 m/s (pessimistic).

5 Completed experiments

All numbers are produced by the code in this project (`results/*.json`); sandbox: 2 CPU cores, no GPU. Test sets are fixed and independent of every generator: 800 everyday scenarios, 800 hazard scenarios (collide if the ego does not react), and the hard-perception subset (633 scenarios with dropout > 0.4 or radius $> 0.6\,\mathrm{m}$).

5.1 E1: Learned policy and diffusion-generated edge cases

Setup. Four DAgger policies on 2 700 scenarios each, 5 seeds: A everyday only; B +1200 guided diffusion; C +1200 unguided diffusion; D +1200 uniform random.

	Collision rate [%]			Hazard
Training data	Everyday	Hazard	Hard-perception	min. gap [m]
A everyday only	4.0 ± 0.4	19.7 ± 1.6	18.9 ± 1.7	1.26 ± 0.04
B + guided diffusion	4.2 ± 1.2	8.2 ± 0.6	8.9 ± 0.7	1.68 ± 0.07
C + unguided diffusion	4.0 ± 0.7	20.2 ± 1.4	19.2 ± 1.0	1.23 ± 0.05
D + uniform random	4.3 ± 0.5	21.3 ± 1.8	20.3 ± 1.7	1.14 ± 0.03
Privileged expert	0.0	0.4	0.5	1.57

Table 3: Mean ± std over 5 seeds.

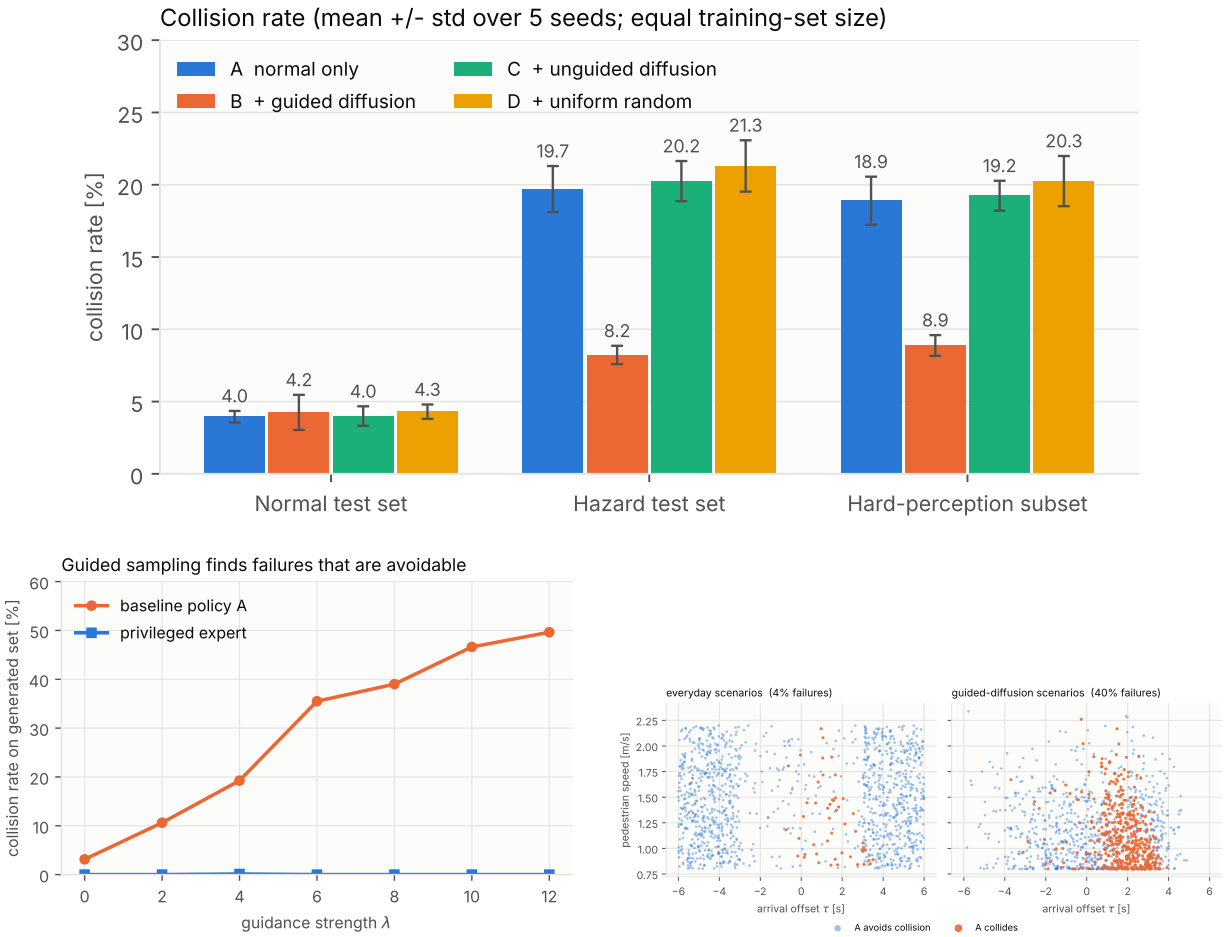


Figure 2: Top: collision rates. Bottom left: guidance strength λ versus failures of policy A on generated sets. Bottom right: where A fails (seed 0).

Observations. Only guided data helps ($19.7 \pm 1.6 \rightarrow 8.2 \pm 0.6\%$ on hazards); unguided or uniform data of the same size does not (C, D within seed noise of A). Guided scenarios are hard for A (38.0% collide) but avoidable (expert 0.1%). Everyday performance and completion are unchanged. Remaining failures are perception-limited.

5.2 E2: NeRF-style digital twin

Setup. 10 expert drives logged (45184 rays), field trained 5000 iterations (247 s); evaluation on 2 unseen drives and on a closed-loop what-if grid (13 pedestrian shifts $\times$ 4 ego speeds) run both in the twin and in the real simulator.

Metric	Value
Mean abs. range error, held-out views, all rays [m]	0.20 (static-only field: 0.40)
Rays within 0.5 m	96.7%
Pedestrian-hit rays (142 of 9024): error [m]	0.77 (static-only: 19.98)
What-if collision outcome agreement (52 cases)	96.2%
Collision rate real / twin	19.2% / 15.4%
Min-gap correlation real vs. twin; mean abs. difference [m]	0.99; 0.10
Twin without dynamic field: collision rate	32.7%

Table 4: Twin fidelity.

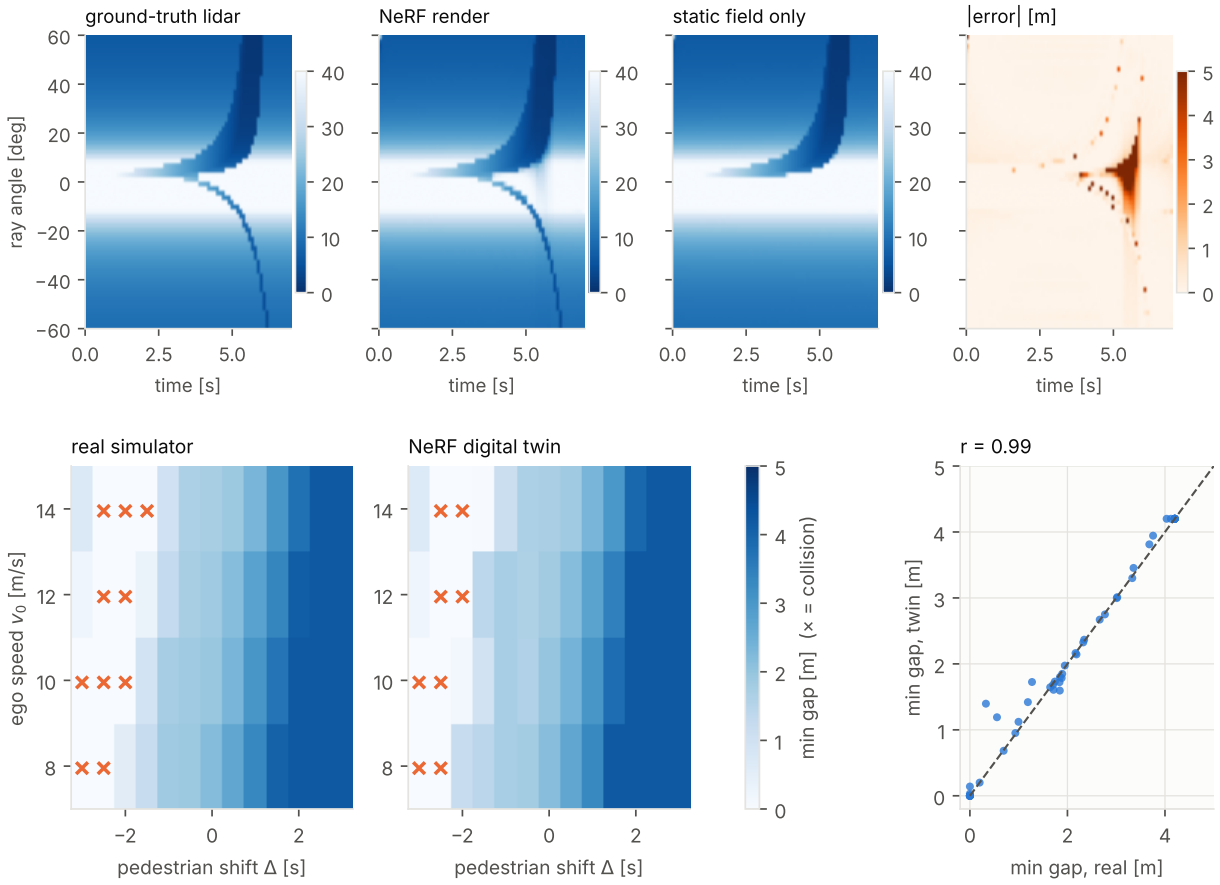


Figure 3: Top: held-out range images (ground truth, render, static-only, error). Bottom: what-if grid, real simulator versus twin (× = collision).

Observations. The dynamic component is essential: without it the pedestrian is invisible and the policy collides 32.7% of the time. The twin misses fine detail on pedestrian rays (the error panel shows the largest errors late in the drive, when the pedestrian is close to the ego) and reproduces only the logged pedestrian trajectory, shifted in time.

5.3 E3: Behaviour-tree verification

Property (checked in every abstract state)	Result
P1 action defined and within actuator limits	all hold
P2 emergency ⇒ full braking $-a_b$	all hold
P3 must-brake ⇒ $a \in [-a_b, -0.3]$	all hold
P4 conflict ⇒ never accelerate	all hold
P5 no conflict ⇒ $a \in [-3, 2.5]$	all hold
P6 acceleration non-increasing in hazard level	all hold

Table 5: Exhaustive check over 32 abstract states (counts per property in `results/stack.json`).

Tree version	Change	Violations in 18000
v1	initial (margins 0/0.5 s, curb pedestrian assumed safe)	78 (`logs/design_iteration_1.log`)
v2	timing margins only (0.5/1.0 s)	276†
v3	+ "yield" level (never accelerate in conflict)	78†
v4	+ curb pedestrian may start at 3.5 m/s	1†
v5 (final)	+ margins 0.3/0.8 s	0 (`results/stack.json`)

Table 6: Bounded model check with true pedestrian state. †Interactive intermediate runs, not archived as files. Of the 78 counterexamples of v1, 75 had a 3.5 m/s pedestrian: the tree accelerated towards a pedestrian waiting at the curb and could not stop when it started running.

Observation. Model checking found a real design flaw that unit tests on everyday data had not. Adding margins alone made it worse, which shows the check is not trivially satisfied. The result covers a finite grid with ideal perception only.

5.4 E4: Planner, optimisation and tracking

Setup. First 100 hazard scenarios, same lidar perception for BT and planner.

Controller	Collision [%]	Completed [%]	Mean min. gap [m]
Learned policy A (seed 0)	23.0	77.0	—
Behaviour tree, lidar perception	2.0	98.0	1.88
Behaviour tree, true pedestrian state	0.0	100.0	1.80
Lattice + optimiser + PD	10.0	90.0	1.92
Lattice + optimiser + PD + BT supervisor	2.0	98.0	2.46

Table 7: Closed-loop comparison. On all 800 hazard / 800 everyday scenarios the BT with lidar collides in 3.2% / 2.4%.

60 hazard scenarios, planner only	RMS tracking error [m]	Collision [%]	95th pct. jerk [m/s^3]
Feed-forward only, nominal brakes	0.068	21.7	13.6
PD + feed-forward, nominal brakes	0.064	23.3	19.0
Feed-forward only, brake gain 0.8	0.073	20.0	12.1
PD + feed-forward, brake gain 0.8	0.068	21.7	16.4
PD + feed-forward, no optimisation	—	21.7	19.2

Table 8: Tracking and optimisation ablations.

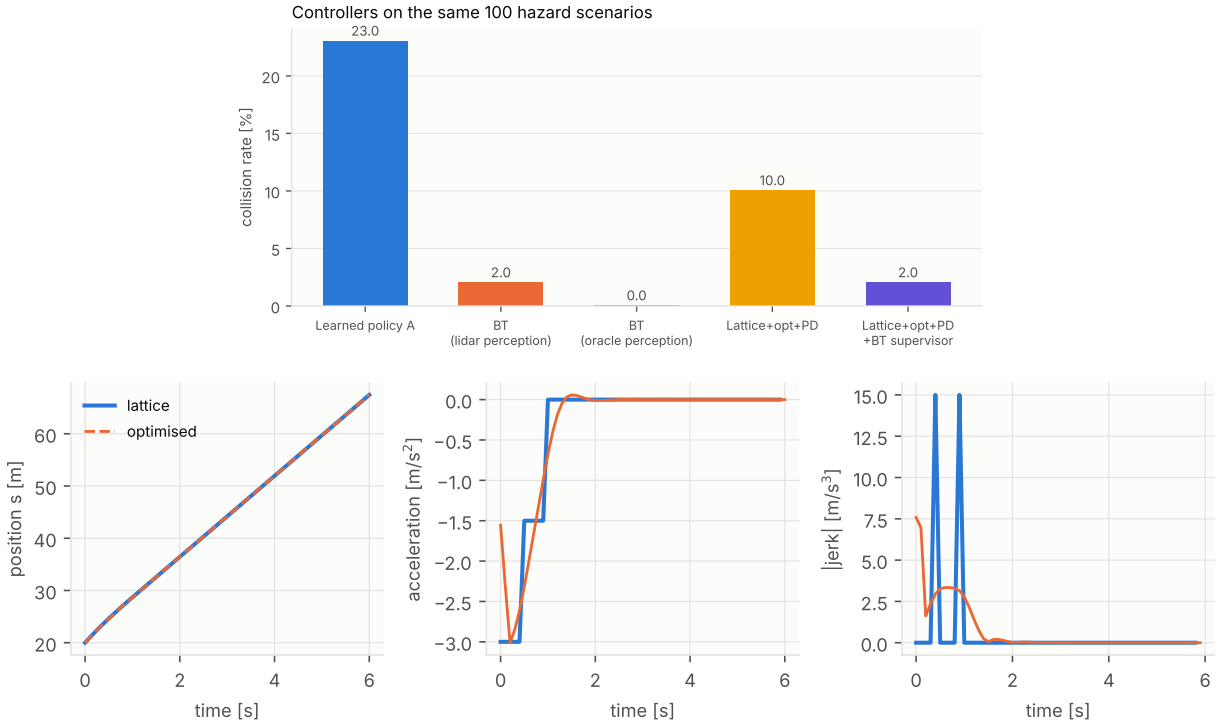


Figure 4: Top: controllers on the same scenarios. Bottom: one lattice path versus its optimised refinement (position, acceleration, jerk).

Observations. (i) The lattice planner alone collides more than the BT because its pedestrian estimate lags a sudden start; the BT supervisor removes most of this at similar completion. (ii) Optimisation reduces the mean per-plan peak jerk from 19.15 to 5.75 m/s^3, but the executed 95th-percentile jerk is essentially unchanged (19.0 with versus 19.2 without optimisation): replanning every 0.5 s restarts the profile and the PD term adds step changes, so the smoothing benefit does not reach the vehicle in this configuration. (iii) PD feedback gave no measurable benefit over feed-forward: replanning at 2 Hz re-anchors the reference to the measured state, so tracking error stays near 6–7 cm either way; the collision differences in the table are within noise for 60 scenarios. (iv) The learned policy is worst, as in E1.

5.5 E5: Fault injection

Fault (400 hazard scenarios)	BT collisions [%]	BT + monitor [%]
None (nominal)	3.0	3.0
Lidar dropout (2 s)	8.0	4.2
Frozen frame (2 s)	4.5	2.0
Range bias ×1.25	1.8	1.8
Latency 0.3 s	2.0	1.5
Brake fade to −4 m/s^2	8.5	8.2

Table 9: Single-fault injection, lidar perception.

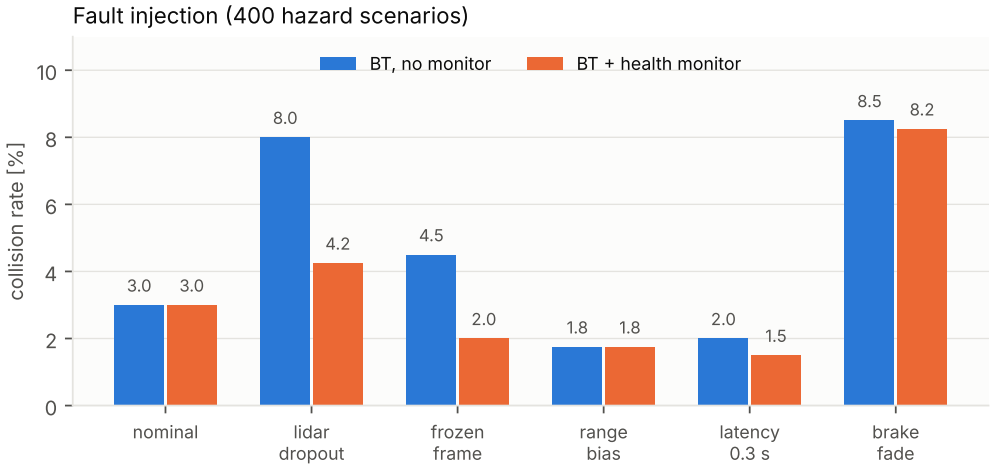


Figure 5: Fault injection results.

Observations. With 400 scenarios the standard error at a 3% rate is about 0.9 percentage points, so differences below roughly 2 points are not significant. The monitor clearly helps for lidar dropout and frozen frames (about halving collisions). Range bias and latency of this size showed no harmful effect. Brake fade is the one fault that raises collisions markedly, and the monitor does not help: it detects fade only after the first hard braking, which is too late. This is recorded as a residual hazard.

5.6 E6: Embedded deployment (host CPU)

Policy (8449 parameters, 8320 MACs)	float32	int8 weights
Latency per inference, gcc -O2, host CPU [µs]	9.0	9.3
Code + weights, gcc -Os [KiB]	33.5	9.2
Max. abs. difference to NumPy output [m/s^2]	2.9e-06	5.7e-06
Collision rate, everyday / hazard set [%]	4.1 / 20.5	4.5 / 22.1

Table 10: C export. Host timings are not microcontroller timings.

Observations. The generated C reproduces the NumPy policy to float rounding. int8 weights cut code+weights size to about a quarter at a small accuracy cost on hazards; latency is not better on the host because the loop converts int8 to float. On-target cycles are not measured (Sec. 8).

5.7 E7: Dataset adapter (software test only)

`datasets.py` turns KITTI tracking labels into scenario vectors (crossing speed, distance, arrival offset τ). It passes a unit test on a synthetic KITTI-format track with known speed (`tests.py`). It has *not* been run on real KITTI, nuScenes or Waymo data, which are not reachable from the sandbox.

6 Safety argument

Scope: the simulated pedestrian-crossing task (operational domain: straight road, one crossing pedestrian, speeds 6–14 m/s, pedestrian speed up to 3.5 m/s, lidar as the only sensor). This is a structured argument linking claims to the evidence above; it is *not* a certified safety case, and no ISO 26262, SOTIF or UL 4600 compliance is claimed.

ID	Claim	Evidence	Residual risk
G0	The ego avoids collision with a crossing pedestrian in the stated domain	G1–G5	see below
G1	The BT logic is well-defined and monotone in hazard	E3 exhaustive check, all properties hold	abstraction of continuous state
G2	With correct perception the BT is safe	E3: 0/18000 violations; 0.0% on hazards	finite grid only
G3	Lidar perception is adequate	E4: BT 2.0% (100 scen.), 3.2% (800) vs. 0% with true state	missed or late detections remain
G4	Sensor faults are detected and handled	E5: dropout and frozen frame roughly halved	latency, brake fade not mitigated
G5	An unverified planner cannot override the BT	E4: planner alone 10.0%, with supervisor 2.0%	supervisor shares the perception
G6	The deployed code equals the tested policy	E6: max. difference 2.9e-06 (float32)	int8 changes hazard collisions

Table 11: Claims and evidence.

ID	Hazard	Mitigation implemented	Status
H1	Collision with crossing pedestrian	BT priority levels, curb assumption, margins	mitigated, residual 3.2%
H2	Pedestrian hidden by parked car	speed cap 8 m/s near occluder	mitigated, not eliminated
H3	Lidar dropout / frozen output	health monitor, speed cap 4 m/s	partly mitigated
H4	Reduced braking	detection by achieved deceleration	**open**
H5	Sensing latency	none	**open**
H6	Planner or learned-policy error	BT supervisor (min of accelerations)	mitigated for tested cases

Table 12: Hazard log.

7 Real-world applications

Technique here	Industrial counterpart
Spacetime field + what-if replay	neural closed-loop sensor simulation of logged drives (e.g. UniSim-style) for regression testing of dangerous events
Guided diffusion over scenarios	adversarial and long-tail scenario generation for perception/planner training and validation
Verified behaviour tree	runtime safety supervisor / fallback layer next to a learned planner (simplex architecture)
Fault injection + monitor	evidence for failure-mode analyses (FMEA/FTA) and degradation strategies
C export, int8 weights	deployment of small policies on automotive-grade controllers

8 Experiments planned for this week

None of these has results yet.

ID	Experiment	Measure	Prerequisite
P1	Run `carla_bt.py` (BT driving a CARLA car, pedestrian crossing)	collision, min gap, comfort vs. this simulator	CARLA ≥0.9.14 server + `pip install carla` (code written, unexecuted)
P2	Fit scenario parameters from KITTI/nuScenes pedestrian tracks (`datasets.py`) and replay in the simulator	distribution of τ, speed, distance vs. synthetic prior	dataset download
P3	Time the C policy on a Cortex-M class board	cycles, latency, RAM/flash	board + toolchain
P4	Repeat E3–E5 with 5 seeds and confidence intervals	mean ± CI for all stack tables	none
P5	Brake-fade mitigation: pre-emptive margin or online decel estimate	collisions under fade	implementation
P6	Latency compensation by state prediction	collisions under 0.3 s latency	implementation
P7	Larger bounded model check (finer grid, perception-error bounds)	violations	none

9 Future experiments, improvements and observations

- **Perception.** The gap between lidar BT (3.2%) and true-state BT (0% on the 800-scenario set) is the largest remaining risk. Ray spacing (about 0.7 m at 20 m) can miss a 0.5 m pedestrian; try denser rays or tracking with a motion model.
- **Twin.** Move to 3-D camera radiance fields or Gaussian splats; add object-centric dynamic fields so that pedestrian speed and path, not only timing, can be edited.
- **Generator.** Guide against the BT as well as the learned policy; add an avoidability filter; extend to multi-agent trajectories.
- **Planner.** Longer open-loop execution between replans would test whether PD feedback matters; add lateral motion.
- **Embedded.** Quantise activations (full int8), measure on target.
- **Observations to keep tracking.** Adding timing margins alone worsened model-check results (E3); a pessimistic default velocity for unseen pedestrians costs some completion time; stack results currently use one seed and 100–400 scenarios, so small differences are not significant.

A Extending this report

1. Write the experiment as `code/run_<name>.py`; save a JSON in `results/` via `paths.RES`.
2. In `code/make_figures_stack.py` add `mac("name", value)` lines (letters only in macro names) and any figure.
3. Copy a block in `sections/04_experiments.tex` (setup, table, figure, observations), move the row in Table 1 and in Sec. 8 from **Planned** to **Completed**.

Reproducibility. All runs are seeded; `results/` and `figures/` shipped with this folder were produced by `make results figures` in a 2-core sandbox (NumPy, Matplotlib). Latencies in E6 depend on the machine.

B Source code

B.1 paths.py

```python
"""Single source of truth for output folders (project root = parent of code/)."""
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RES, FIG, LOG, BUILD = (ROOT / d for d in ("results", "figures", "logs", "build"))
for _d in (RES, FIG, LOG, BUILD):
    _d.mkdir(exist_ok=True)
```

B.2 nn.py

```python
"""Tiny neural-network toolkit in pure NumPy (manual back-propagation + Adam).

Karpathy-style: no deep-learning framework, every gradient is written out, so the
whole project runs anywhere NumPy runs (laptop, Colab, Jupyter).
"""
import numpy as np


def _act(z, kind):
    if kind == "relu":
        return np.maximum(z, 0.0)
    if kind == "silu":
        return z / (1.0 + np.exp(-z))
    raise ValueError(kind)


def _dact(z, kind):
    if kind == "relu":
        return (z > 0).astype(z.dtype)
    s = 1.0 / (1.0 + np.exp(-z))
    return s * (1.0 + z * (1.0 - s))


class MLP:
    """Fully-connected network: sizes=[in, h1, ..., out]. Linear output layer."""

    def __init__(self, sizes, rng, act="relu", out_scale=1.0):
        self.sizes, self.act = list(sizes), act
        self.W, self.b = [], []
        for i in range(len(sizes) - 1):
            W = rng.standard_normal((sizes[i], sizes[i + 1])) * np.sqrt(2.0 / sizes[i])
            self.W.append(W.astype(np.float32))
            self.b.append(np.zeros(sizes[i + 1], np.float32))
        self.W[-1] *= out_scale
        self.m = [np.zeros_like(p) for p in self.W + self.b]
        self.v = [np.zeros_like(p) for p in self.W + self.b]
        self.t = 0

    # ------------------------------------------------------------------ forward
    def forward(self, x):
        h = np.asarray(x, np.float32)
        self.xs, self.zs = [], []
        L = len(self.W)
        for i in range(L):
            self.xs.append(h)
            z = h @ self.W[i] + self.b[i]
            if i < L - 1:
                self.zs.append(z)
                h = _act(z, self.act)
            else:
                h = z
        return h

    # ----------------------------------------------------------------- backward
    def backward(self, dy):
        """Back-propagate dL/d(output); stores parameter grads, returns dL/d(input)."""
```

```
        L = len(self.W)
        self.gW, self.gb = [None] * L, [None] * L
        g = np.asarray(dy, np.float32)
        for i in reversed(range(L)):
            self.gW[i] = self.xs[i].T @ g
            self.gb[i] = g.sum(0)
            g = g @ self.W[i].T
            if i > 0:
                g = g * _dact(self.zs[i - 1], self.act)
        return g

    # ------------------------------------------------------------------ Adam
    def step(self, lr, b1=0.9, b2=0.999, eps=1e-8, clip=None):
        self.t += 1
        params, grads = self.W + self.b, self.gW + self.gb
        if clip is not None:
            n = np.sqrt(sum(float((g ** 2).sum()) for g in grads))
            if n > clip:
                grads = [g * (clip / n) for g in grads]
        for k, (p, g) in enumerate(zip(params, grads)):
            self.m[k] = b1 * self.m[k] + (1 - b1) * g
            self.v[k] = b2 * self.v[k] + (1 - b2) * g * g
            mh = self.m[k] / (1 - b1 ** self.t)
            vh = self.v[k] / (1 - b2 ** self.t)
            p -= lr * mh / (np.sqrt(vh) + eps)

    # ------------------------------------------------------------------ misc
    def save(self, path):
        np.savez(path, **{f"W{i}": w for i, w in enumerate(self.W)},
                 **{f"b{i}": b for i, b in enumerate(self.b)})

    def load(self, path):
        d = np.load(path)
        for i in range(len(self.W)):
            self.W[i][...] = d[f"W{i}"]
            self.b[i][...] = d[f"b{i}"]


class Adam:
    """Adam for a single dense array (used for the NeRF feature grids)."""

    def __init__(self, p, b1=0.9, b2=0.999, eps=1e-8):
        self.p, self.b1, self.b2, self.eps = p, b1, b2, eps
        self.m, self.v, self.t = np.zeros_like(p), np.zeros_like(p), 0

    def step(self, g, lr):
        self.t += 1
        self.m = self.b1 * self.m + (1 - self.b1) * g
        self.v = self.b2 * self.v + (1 - self.b2) * g * g
        mh = self.m / (1 - self.b1 ** self.t)
        vh = self.v / (1 - self.b2 ** self.t)
        self.p -= lr * mh / (np.sqrt(vh) + self.eps)
```

B.3 `sim.py`

```
"""A small, fully vectorised 2-D driving simulator (pedestrian-crossing world).

World (bird's-eye view, ego drives along +x):

    y=+7 ===================== building line =====================
    y=+5.5 [ped waits here] (hidden behind parked car)
    y=+4.0 [parked car] y=+2.2
    y= 0 ego ------------------------------------------>
    y=-5.5 [ped ends here]
    y=-7 ===================== building line =====================

A pedestrian crosses the road at x = dist. The ego car (longitudinal control only)
has a 64-ray lidar. Everything is batched over scenarios, so thousands of
episodes run at once in NumPy.

Scenario vector z = (v0, dist, ped_speed, tau, occl, ped_r, dropout, noise)
    v0 initial ego speed [m/s]
```

```
    dist x-position of the crossing [m]
    ped_speed pedestrian walking speed [m/s]
    tau the pedestrian reaches the lane centre tau seconds after the ego
             would arrive there at constant speed (tau ~ 0 -> collision course)
    occl parked-car occluder size (0 = none, 1 = 6 m long)
    ped_r pedestrian footprint radius [m] (0.3 normal, up to 1.0 = odd costume)
    dropout probability that a lidar ray on the pedestrian is lost (dark clothes)
    noise lidar range noise std [m]
"""
import numpy as np

DT = 0.1
T_MAX = 18.0
N_STEPS = int(T_MAX / DT)
EGO_L, EGO_W = 4.5, 1.9
A_MIN, A_MAX = -8.0, 3.0
N_RAYS = 64
FOV = np.deg2rad(120.0)
R_MAX = 40.0
SENSOR_DX = 2.0
WALL_Y = 7.0
PED_Y0, PED_Y1 = 5.5, -5.5
PC_Y0, PC_Y1, PC_GAP = 2.2, 4.0, 3.0

PNAMES = ["v0", "dist", "ped_speed", "tau", "occl", "ped_r", "dropout", "noise"]
LO = np.array([6.0, 35.0, 0.8, -6.0, 0.0, 0.25, 0.0, 0.02])
HI = np.array([14.0, 70.0, 3.5, 6.0, 1.0, 1.0, 0.8, 0.30])
ANG = np.linspace(-FOV / 2, FOV / 2, N_RAYS)
COS, SIN = np.cos(ANG), np.sin(ANG)


# ------------------------------------------------------------------ scenario priors
def sample_normal(rng, n):
    """'Everyday' driving distribution: hazards and odd pedestrians are rare."""
    v0 = rng.uniform(7, 13, n)
    dist = rng.uniform(35, 70, n)
    s = rng.uniform(0.8, 2.2, n)
    crit = rng.random(n) < 0.15
    tau = np.where(crit, rng.uniform(-3, 3, n), rng.choice([-1.0, 1.0], n) * rng.uniform(3, 6, n))
    occl = rng.uniform(0, 1, n)
    big = rng.random(n) < 0.05
    pr = np.where(big, rng.uniform(0.4, 1.0, n), rng.uniform(0.25, 0.4, n))
    drop = np.where(rng.random(n) < 0.10, rng.uniform(0, 0.3, n), 0.0)
    noise = rng.uniform(0.02, 0.10, n)
    return np.stack([v0, dist, s, tau, occl, pr, drop, noise], 1)


def sample_uniform(rng, n):
    return rng.uniform(LO, HI, (n, len(LO)))


# ------------------------------------------------------------------ geometry
def ped_traj(s, t_start, t):
    y = np.clip(PED_Y0 - s * (t - t_start), PED_Y1, PED_Y0)
    moving = (t > t_start) & (y > PED_Y1)
    return y, np.where(moving, -s, 0.0)


def ped_t_start(P):
    v0, dist, s, tau = P[:, 0], P[:, 1], P[:, 2], P[:, 3]
    return dist / v0 + tau - PED_Y0 / s


def rect_circle_gap(ex, ey, px, py, pr):
    dx = np.maximum(np.abs(px - ex) - EGO_L / 2, 0.0)
    dy = np.maximum(np.abs(py - ey) - EGO_W / 2, 0.0)
    return np.hypot(dx, dy) - pr


def open_loop_gap(P):
    """Minimum ego-pedestrian gap if the ego simply keeps its speed (no reaction)."""
    t = np.arange(0, T_MAX, DT)[None, :]
    v0 = P[:, 0:1]
    ts = ped_t_start(P)[:, None]
    y, _ = ped_traj(P[:, 2:3], ts, t)
```

```
    g = rect_circle_gap(v0 * t, 0.0, P[:, 1:2], y, P[:, 5:6])
    return np.maximum(g.min(1), 0.0)


# ------------------------------------------------------------------- batch simulator
class BatchSim:
    def __init__(self, P, rng, t_start=None, ey=0.0, v0=None):
        self.P, self.rng = np.asarray(P, float), rng
        B = len(self.P)
        self.B = B
        self.xp, self.s, self.pr = self.P[:, 1].copy(), self.P[:, 2].copy(), self.P[:, 5].copy()
        self.drop, self.noise = self.P[:, 6].copy(), self.P[:, 7].copy()
        self.t_start = ped_t_start(self.P) if t_start is None else np.asarray(t_start, float)
        self.pc_len = np.where(self.P[:, 4] > 0.1, 6.0 * self.P[:, 4], 0.0)
        self.pc_x1 = self.xp - PC_GAP
        self.pc_x0 = self.pc_x1 - self.pc_len
        self.x = np.zeros(B)
        self.v = (self.P[:, 0] if v0 is None else np.asarray(v0, float)).copy()
        self.ey = np.broadcast_to(np.asarray(ey, float), (B,)).copy()
        self.t, self.k = 0.0, 0
        self.done = np.zeros(B, bool)
        self.collided = np.zeros(B, bool)
        self.passed = np.zeros(B, bool)
        self.min_gap = np.full(B, np.inf)
        self.prev = None
        self.lid = None

    def ped_state(self, t=None):
        return ped_traj(self.s, self.t_start, self.t if t is None else t)

    # ------------------------------------------------------------------- lidar
    def lidar(self):
        B = self.B
        ox, oy = (self.x + SENSOR_DX)[:, None], self.ey[:, None]
        dx, dy = COS[None, :], SIN[None, :]
        tw = np.where(dy > 0, (WALL_Y - oy) / dy, (-WALL_Y - oy) / dy)
        # parked car (axis-aligned box, slab method)
        t1, t2 = (self.pc_x0[:, None] - ox) / dx, (self.pc_x1[:, None] - ox) / dx
        u1, u2 = (PC_Y0 - oy) / dy, (PC_Y1 - oy) / dy
        tmin = np.maximum(np.minimum(t1, t2), np.minimum(u1, u2))
        tmax = np.minimum(np.maximum(t1, t2), np.maximum(u1, u2))
        hit = (tmax >= np.maximum(tmin, 0)) & (self.pc_len[:, None] > 0)
        tp = np.where(hit, np.maximum(tmin, 0.0), np.inf)
        # pedestrian (circle)
        py, _ = self.ped_state()
        cx, cy = self.xp[:, None] - ox, py[:, None] - oy
        b = dx * cx + dy * cy
        c0 = cx ** 2 + cy ** 2 - (self.pr[:, None] ** 2)
        disc = b * b - c0
        tc = np.where(disc >= 0, b - np.sqrt(np.maximum(disc, 0)), np.inf)
        tc = np.where(tc > 0, tc, np.where(c0 < 0, 0.0, np.inf))
        lost = self.rng.random((B, N_RAYS)) < self.drop[:, None]
        tc = np.where(lost, np.inf, tc)
        r = np.minimum(np.minimum(tw, tp), np.minimum(tc, R_MAX))
        r = r + self.rng.standard_normal((B, N_RAYS)) * self.noise[:, None]
        return np.clip(r, 0.0, R_MAX)

    def observe(self):
        self.lid = self.lidar()
        if self.prev is None:
            self.prev = self.lid.copy()
        obs = make_obs(self.lid, self.prev, self.v)
        self.prev = self.lid
        return obs

    # ------------------------------------------------------------------- step
    def step(self, a):
        act = ~self.done
        a = np.clip(a, A_MIN, A_MAX)
        v_new = np.maximum(self.v + a * DT, 0.0)
        self.v = np.where(act, v_new, self.v)
        self.x = np.where(act, self.x + self.v * DT, self.x)
        self.t += DT
        self.k += 1
        y, _ = self.ped_state()
```

```
        gap = rect_circle_gap(self.x, self.ey, self.xp, y, self.pr)
        self.min_gap = np.where(act, np.minimum(self.min_gap, gap), self.min_gap)
        col = act & (gap <= 0)
        self.collided |= col
        psd = act & ((self.x - EGO_L / 2) > self.xp + self.pr + 1.0)
        self.passed |= psd
        self.done |= col | psd
        return act


def make_obs(lid, prev, v):
    """Policy input: min-pooled lidar (32), its temporal difference (32), speed."""
    a = lid.reshape(len(lid), N_RAYS // 2, 2).min(2) / R_MAX
    b = prev.reshape(len(prev), N_RAYS // 2, 2).min(2) / R_MAX
    return np.concatenate([a, (a - b) * 10.0, (v / 15.0)[:, None]], 1).astype(np.float32)
```

B.4 driving.py

```
"""Expert controller, DAgger imitation learning and evaluation (all NumPy)."""
import numpy as np
from nn import MLP
from sim import (BatchSim, N_STEPS, EGO_L, EGO_W, A_MIN, A_MAX, make_obs)

OBS_DIM = 65
ACT_SCALE = 8.0


# ------------------------------------------------------------------- privileged expert
def expert_action(sim):
    """Rule-based expert that sees the true pedestrian state (privileged information)."""
    x, v = sim.x, sim.v
    xf, rear = x + EGO_L / 2, x - EGO_L / 2
    y, vy = sim.ped_state()
    band = EGO_W / 2 + sim.pr + 0.4
    xs = sim.xp - sim.pr - 0.5 # stop line for the ego front
    xc = sim.xp + sim.pr + 0.5 # line the ego rear must have passed
    moving = vy < 0
    in_band = np.abs(y) < band
    with np.errstate(divide="ignore", invalid="ignore"):
        t_enter = np.where(moving, np.maximum((y - band) / (-vy), 0.0), 0.0)
        t_leave = np.where(moving, np.maximum((y + band) / (-vy), 0.0), np.inf)
    static_out = (~moving) & (~in_band)
    past = moving & (y < -band)
    clears_before = (rear + v * t_enter) > xc
    with np.errstate(invalid="ignore"):
        gone_first = np.where(np.isinf(t_leave), False, (xf + v * (t_leave + 0.5)) < xs)
    conflict = ~(static_out | past | clears_before | gone_first)
    d = xs - xf
    a_req = v ** 2 / (2.0 * np.maximum(d, 0.2))
    brake = -np.clip(1.3 * a_req + 0.3, 0.0, 8.0)
    brake = np.where(d <= 0, -8.0, brake)
    v_des = np.full_like(v, 12.0)
    ahead = (sim.pc_len > 0) & (sim.pc_x0 - xf < 35.0) & (xf < sim.pc_x1)
    v_des = np.where(ahead, 8.0, v_des) # defensive driving near occluders
    a_c = np.clip(1.0 * (v_des - v), -3.0, 2.5)
    return np.where(conflict & ((a_req > 0.8) | (d <= 0)), np.minimum(a_c, brake), a_c)


# ------------------------------------------------------------------------ policy
def policy_action(net, obs):
    return np.clip(net.forward(obs)[:, 0] * ACT_SCALE, A_MIN, A_MAX)


def collect(P, net, beta, rng, act_noise=0.4):
    """Roll out (a mixture of expert and learner) and label every state with the expert."""
    sim = BatchSim(P, rng)
    O, Y = [], []
    for _ in range(N_STEPS):
        obs = sim.observe()
        a_exp = expert_action(sim)
        if net is None:
            a = a_exp
```

```
        else:
            use_exp = rng.random(sim.B) < beta
            a = np.where(use_exp, a_exp, policy_action(net, obs))
        act = ~sim.done
        O.append(obs[act])
        Y.append(a_exp[act])
        sim.step(a + rng.standard_normal(sim.B) * act_noise)
        if sim.done.all():
            break
    return np.concatenate(O), np.concatenate(Y).astype(np.float32)


def fit(net, O, Y, iters, rng, lr=2e-3, bs=512):
    losses = []
    for i in range(iters):
        idx = rng.integers(0, len(O), bs)
        pred = net.forward(O[idx])[:, 0]
        err = pred - Y[idx] / ACT_SCALE
        net.backward((2.0 * err / bs)[:, None])
        net.step(lr * (0.5 * (1 + np.cos(np.pi * i / iters)) * 0.9 + 0.1), clip=5.0)
        if i % 100 == 0:
            losses.append(float((err ** 2).mean()))
    return losses


def train_policy(P, seed, rounds=3, iters=2500, hidden=64, verbose=False):
    """DAgger: round 0 = expert only, later rounds mix in the learner (beta 0.5, 0)."""
    rng = np.random.default_rng(seed)
    net = MLP([OBS_DIM, hidden, hidden, 1], rng, out_scale=0.1)
    betas = [1.0, 0.5, 0.0, 0.0][:rounds]
    O, Y, curve = None, None, []
    for r, beta in enumerate(betas):
        o, y = collect(P, None if r == 0 else net, beta, rng)
        O = o if O is None else np.concatenate([O, o])
        Y = y if Y is None else np.concatenate([Y, y])
        curve += fit(net, O, Y, iters, rng)
        if verbose:
            print(f" dagger round {r}: {len(O)} samples, loss {curve[-1]:.4f}")
    net.curve = curve
    return net


# ------------------------------------------------------------------- evaluation
def evaluate(P, controller, seed=0, t_start=None):
    """controller: 'expert' or an MLP policy. Returns per-scenario metrics."""
    rng = np.random.default_rng(seed)
    sim = BatchSim(P, rng, t_start=t_start)
    for _ in range(N_STEPS):
        obs = sim.observe()
        a = expert_action(sim) if isinstance(controller, str) else policy_action(controller, obs)
        sim.step(a)
        if sim.done.all():
            break
    return dict(collided=sim.collided.copy(), passed=sim.passed.copy(),
                min_gap=np.minimum(sim.min_gap, 20.0), x_final=sim.x.copy())
```

B.5 scenario_gen.py

```
"""Generative AI for edge cases: a small DDPM over scenario vectors + failure guidance.

1. Train a denoising-diffusion model on 'logged' everyday scenarios (it learns the prior).
2. Train a noise-aware surrogate g(x_t, t) that predicts how close the *current driving
   policy* comes to the pedestrian (minimum gap) in that scenario.
3. Sample with classifier guidance: eps' = eps + lam * sqrt(1 - abar_t) * grad g
   which tilts the prior towards scenarios where the policy has small gaps / collides.
"""
import numpy as np
from nn import MLP
from sim import LO, HI, PNAMES

T_DIFF = 200
BETAS = np.linspace(1e-4, 0.05, T_DIFF)
```

```
ALPHAS = 1.0 - BETAS
ABAR = np.cumprod(ALPHAS)
D = len(PNAMES)
TFREQ = np.array([1, 2, 4, 8, 16, 32, 64, 128], float)


def normalize(P):
    return (P - LO) / (HI - LO) * 2.0 - 1.0


def denormalize(X):
    return (np.clip(X, -1, 1) + 1.0) / 2.0 * (HI - LO) + LO


def t_embed(t):
    u = (np.asarray(t, float) / T_DIFF)[:, None] * TFREQ[None, :] * np.pi
    return np.concatenate([np.sin(u), np.cos(u)], 1).astype(np.float32)


def noisy(x0, t, rng):
    eps = rng.standard_normal(x0.shape).astype(np.float32)
    ab = ABAR[t][:, None]
    return (np.sqrt(ab) * x0 + np.sqrt(1 - ab) * eps).astype(np.float32), eps


class ScenarioDiffusion:
    def __init__(self, seed=0, hidden=128):
        self.rng = np.random.default_rng(seed)
        self.eps_net = MLP([D + 16, hidden, hidden, hidden, D], self.rng, act="silu", out_scale=0.5)
        self.surr = None
        self.loss_curve, self.surr_curve = [], []

    # ------------------------------------------------------------- prior training
    def fit_prior(self, P, iters=6000, bs=512, lr=1e-3):
        X = normalize(P).astype(np.float32)
        for i in range(iters):
            x0 = X[self.rng.integers(0, len(X), bs)]
            t = self.rng.integers(0, T_DIFF, bs)
            xt, eps = noisy(x0, t, self.rng)
            pred = self.eps_net.forward(np.concatenate([xt, t_embed(t)], 1))
            err = pred - eps
            self.eps_net.backward(2.0 * err / (bs * D))
            self.eps_net.step(lr * (0.5 * (1 + np.cos(np.pi * i / iters)) * 0.95 + 0.05), clip=5.0)
            if i % 100 == 0:
                self.loss_curve.append(float((err ** 2).mean()))

    # ---------------------------------------------------------- surrogate training
    def fit_surrogate(self, P, y, iters=6000, bs=512, lr=1e-3, hidden=128):
        """y in [0,1]: normalised minimum gap of the current policy (0 = collision)."""
        self.surr = MLP([D + 16, hidden, hidden, 1], self.rng, act="silu", out_scale=0.3)
        X = normalize(P).astype(np.float32)
        y = y.astype(np.float32)
        for i in range(iters):
            idx = self.rng.integers(0, len(X), bs)
            t = self.rng.integers(0, T_DIFF, bs)
            xt, _ = noisy(X[idx], t, self.rng)
            pred = self.surr.forward(np.concatenate([xt, t_embed(t)], 1))[:, 0]
            err = pred - y[idx]
            self.surr.backward((2.0 * err / bs)[:, None])
            self.surr.step(lr * (0.5 * (1 + np.cos(np.pi * i / iters)) * 0.95 + 0.05), clip=5.0)
            if i % 100 == 0:
                self.surr_curve.append(float((err ** 2).mean()))

    # ------------------------------------------------------------------ sampling
    def sample(self, n, guidance=0.0, seed=0):
        rng = np.random.default_rng(seed)
        x = rng.standard_normal((n, D)).astype(np.float32)
        for t in reversed(range(T_DIFF)):
            tt = np.full(n, t)
            inp = np.concatenate([x, t_embed(tt)], 1)
            eps = self.eps_net.forward(inp)
            if guidance > 0 and self.surr is not None:
                self.surr.forward(inp)
                dinp = self.surr.backward(np.ones((n, 1), np.float32))[:, :D]
                eps = eps + guidance * np.sqrt(1 - ABAR[t]) * dinp
```

```
            x0 = np.clip((x - np.sqrt(1 - ABAR[t]) * eps) / np.sqrt(ABAR[t]), -1, 1)
            ab_prev = ABAR[t - 1] if t > 0 else 1.0
            mean = (np.sqrt(ab_prev) * BETAS[t] / (1 - ABAR[t])) * x0 \
                + (np.sqrt(ALPHAS[t]) * (1 - ab_prev) / (1 - ABAR[t])) * x
            if t > 0:
                var = BETAS[t] * (1 - ab_prev) / (1 - ABAR[t])
                x = (mean + np.sqrt(var) * rng.standard_normal((n, D))).astype(np.float32)
            else:
                x = mean.astype(np.float32)
        return denormalize(x)
```

B.6 nerf2d.py

```
"""NeRF-style 'digital twin' of a logged drive, in 2-D, written from scratch in NumPy.

A neural radiance field normally maps (x, y, z, view) -> (colour, density) and is trained
by differentiable volume rendering of camera pixels. Here the sensor is a lidar and the
world is 2-D, so the field predicts only *density*, and a ray is 'rendered' into an
expected hit range:

    D(ray) = sum_i w_i r_i + T_N * R_max , w_i = T_{i-1} (1 - exp(-sigma_i delta_i))

The scene is split, as in D-NeRF / dynamic NeRFs, into
    sigma(x, y, t) = sigma_static(x, y) + sigma_dynamic(x, y, t)
Each part is a learnable feature grid (bilinear / trilinear interpolation, Instant-NGP
style) followed by a small MLP decoder. Because time is an explicit input of the dynamic
part, we can re-render the same drive with the pedestrian shifted in time (a 'what-if').
All gradients (volume rendering, interpolation, decoders) are derived by hand.
"""
import numpy as np
from nn import MLP, Adam
from sim import (R_MAX, ANG, COS, SIN, N_RAYS, BatchSim, SENSOR_DX, N_STEPS, make_obs)

X0, X1, Y0, Y1 = 0.0, 100.0, -8.0, 8.0
CELL, TCELL, C = 0.25, 0.2, 4
SCALE = 20.0
N_SAMP = 128


def softplus(z):
    return np.logaddexp(0.0, z)


def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))


class NeuralField:
    def __init__(self, x_ped, t_max=12.0, seed=0):
        rng = np.random.default_rng(seed)
        self.rng, self.t_max = rng, t_max
        self.nx, self.ny = int((X1 - X0) / CELL) + 1, int((Y1 - Y0) / CELL) + 1
        self.wx0, self.wx1 = x_ped - 10.0, x_ped + 10.0 # dynamic window
        self.nxd, self.nt = int((self.wx1 - self.wx0) / CELL) + 1, int(t_max / TCELL) + 1
        self.Fs = (rng.standard_normal((self.nx * self.ny, C)) * 0.1).astype(np.float32)
        self.Fd = (rng.standard_normal((self.nxd * self.ny * self.nt, C)) * 0.1).astype(np.float32)
        self.dec_s = MLP([C, 16, 1], rng, out_scale=0.5)
        self.dec_d = MLP([C, 16, 1], rng, out_scale=0.5)
        self.dec_s.b[-1][:] = -6.0
        self.dec_d.b[-1][:] = -8.0
        self.opt_s, self.opt_d = Adam(self.Fs), Adam(self.Fd)
        self.loss_curve = []

    # ------------------------------------------------------------------ interpolation
    def _static_idx(self, x, y):
        fx = np.clip((x - X0) / CELL, 0, self.nx - 1 - 1e-6)
        fy = np.clip((y - Y0) / CELL, 0, self.ny - 1 - 1e-6)
        ix, iy = fx.astype(np.int64), fy.astype(np.int64)
        ax, ay = (fx - ix).astype(np.float32), (fy - iy).astype(np.float32)
        idx, w = [], []
        for dx in (0, 1):
            for dy in (0, 1):
```

```
            idx.append((ix + dx) * self.ny + (iy + dy))
            w.append((ax if dx else 1 - ax) * (ay if dy else 1 - ay))
    mask = ((x >= X0) & (x <= X1) & (y >= Y0) & (y <= Y1)).astype(np.float32)
    return np.stack(idx, 1), np.stack(w, 1), mask

def _dyn_idx(self, x, y, t):
    fx = np.clip((x - self.wx0) / CELL, 0, self.nxd - 1 - 1e-6)
    fy = np.clip((y - Y0) / CELL, 0, self.ny - 1 - 1e-6)
    ft = np.clip(t / TCELL, 0, self.nt - 1 - 1e-6)
    ix, iy, it = fx.astype(np.int64), fy.astype(np.int64), ft.astype(np.int64)
    ax, ay, at = (fx - ix).astype(np.float32), (fy - iy).astype(np.float32), (ft - it).astype(np.
        float32)
    idx, w = [], []
    for dx in (0, 1):
        for dy in (0, 1):
            for dt in (0, 1):
                idx.append(((ix + dx) * self.ny + (iy + dy)) * self.nt + (it + dt))
                w.append((ax if dx else 1 - ax) * (ay if dy else 1 - ay) * (at if dt else 1 - at))
    mask = ((x >= self.wx0) & (x <= self.wx1) & (y >= Y0) & (y <= Y1)).astype(np.float32)
    return np.stack(idx, 1), np.stack(w, 1), mask

# ------------------------------------------------------------------ density
def density(self, x, y, t, keep=False, use_dyn=True):
    """sigma(x,y,t) for flat arrays of points. keep=True stores caches for backward."""
    out = {}
    si, sw, sm = self._static_idx(x, y)
    fs = (self.Fs[si] * sw[..., None]).sum(1)
    os_ = self.dec_s.forward(fs)[:, 0]
    sig = SCALE * softplus(os_) * sm
    out["s"] = (si, sw, sm, os_)
    if use_dyn:
        di, dw, dm = self._dyn_idx(x, y, t)
        fd = (self.Fd[di] * dw[..., None]).sum(1)
        od = self.dec_d.forward(fd)[:, 0]
        sig_d = SCALE * softplus(od) * dm
        sig = sig + sig_d
        out["d"] = (di, dw, dm, od)
        out["sig_d"] = sig_d
    if keep:
        self.cache = out
    return sig

def density_backward(self, dsig, dsig_d_extra=None):
    si, sw, sm, os_ = self.cache["s"]
    g = (dsig * SCALE * sigmoid(os_) * sm)[:, None].astype(np.float32)
    dfs = self.dec_s.backward(g)
    gFs = np.zeros_like(self.Fs)
    for c in range(C):
        gFs[:, c] = np.bincount(si.ravel(), weights=(sw * dfs[:, c:c + 1]).ravel(), minlength=len(self.
            Fs))
    di, dw, dm, od = self.cache["d"]
    dd = dsig if dsig_d_extra is None else dsig + dsig_d_extra
    g = (dd * SCALE * sigmoid(od) * dm)[:, None].astype(np.float32)
    dfd = self.dec_d.backward(g)
    gFd = np.zeros_like(self.Fd)
    for c in range(C):
        gFd[:, c] = np.bincount(di.ravel(), weights=(dw * dfd[:, c:c + 1]).ravel(), minlength=len(self.
            Fd))
    return gFs, gFd

# ------------------------------------------------------------------ rendering
def _ray_points(self, ox, oy, ang, jitter):
    n = N_SAMP
    edges = np.linspace(0.0, R_MAX, n + 1)
    r0 = edges[:-1][None, :] + (self.rng.random((len(ox), n)) if jitter else 0.5) * (R_MAX / n)
    r = np.broadcast_to(r0, (len(ox), n))
    px = ox[:, None] + r * np.cos(ang)[:, None]
    py = oy[:, None] + r * np.sin(ang)[:, None]
    delta = np.concatenate([r[:, 1:] - r[:, :-1], (R_MAX - r[:, -1:])], 1)
    return r, px, py, delta

def render(self, ox, oy, ang, t, jitter=False, use_dyn=True, chunk=40000):
    R = len(ox)
    per = max(1, chunk // N_SAMP)
    out = np.empty(R)
```

```
        for s in range(0, R, per):
            sl = slice(s, s + per)
            r, px, py, delta = self._ray_points(ox[sl], oy[sl], ang[sl], jitter)
            tt = np.broadcast_to(t[sl][:, None], px.shape)
            sig = self.density(px.ravel(), py.ravel(), tt.ravel(), use_dyn=use_dyn).reshape(px.shape)
            E = np.exp(-np.cumsum(sig * delta, 1))
            g = np.concatenate([r[:, 1:] - r[:, :-1], R_MAX - r[:, -1:]], 1)
            out[sl] = r[:, 0] + (E * g).sum(1)
        return out

    # ------------------------------------------------------------------ training
    def fit(self, rays, iters=4000, bs=192, lr_grid=2e-2, lr_dec=2e-3, l1_dyn=0.5, verbose=True):
        ox, oy, ang, t, gt = rays
        N = len(ox)
        for it in range(iters):
            decay = 0.5 * (1 + np.cos(np.pi * it / iters)) * 0.95 + 0.05
            idx = self.rng.integers(0, N, bs)
            r, px, py, delta = self._ray_points(ox[idx], oy[idx], ang[idx], True)
            tt = np.broadcast_to(t[idx][:, None], px.shape)
            sig = self.density(px.ravel(), py.ravel(), tt.ravel(), keep=True).reshape(px.shape)
            E = np.exp(-np.cumsum(sig * delta, 1))
            g = np.concatenate([r[:, 1:] - r[:, :-1], R_MAX - r[:, -1:]], 1)
            D = r[:, 0] + (E * g).sum(1)
            err = D - gt[idx]
            loss = float(np.where(np.abs(err) < 1, 0.5 * err ** 2, np.abs(err) - 0.5).mean())
            dD = np.clip(err, -1, 1) / bs
            tail = np.cumsum((E * g)[:, ::-1], 1)[:, ::-1] # sum_{i>=k} E_i g_i
            dsig = (-delta * tail * dD[:, None]).ravel()
            reg = np.full(dsig.shape, l1_dyn / dsig.size)
            gFs, gFd = self.density_backward(dsig, reg)
            self.opt_s.step(gFs, lr_grid * decay)
            self.opt_d.step(gFd, lr_grid * decay)
            self.dec_s.step(lr_dec * decay, clip=5.0)
            self.dec_d.step(lr_dec * decay, clip=5.0)
            if it % 50 == 0:
                self.loss_curve.append(loss)
            if verbose and it % 500 == 0:
                print(f" nerf iter {it:5d} huber loss {loss:.4f}", flush=True)

    # ------------------------------------------------------------------ BEV map
    def bev(self, xs, ys, t, which="both"):
        X, Y = np.meshgrid(xs, ys)
        tt = np.full(X.size, t)
        sig = self.density(X.ravel(), Y.ravel(), tt, use_dyn=(which != "static"))
        if which == "dyn":
            sig = self.density(X.ravel(), Y.ravel(), tt)
            sig = sig - self.density(X.ravel(), Y.ravel(), tt, use_dyn=False)
        return sig.reshape(X.shape)


# ============================================================================ logging
def run_logged_drives(P, t_start, ey, v0, seed):
    """Drive the privileged expert through a fixed scene and record all lidar returns."""
    from driving import expert_action
    rng = np.random.default_rng(seed)
    sim = BatchSim(P, rng, t_start=t_start, ey=ey, v0=v0)
    rec = dict(ox=[], oy=[], t=[], rng=[], drive=[], step=[])
    for k in range(N_STEPS):
        lid = sim.lidar()
        act = ~sim.done
        rec["ox"].append((sim.x + SENSOR_DX)[act]); rec["oy"].append(sim.ey[act])
        rec["t"].append(np.full(act.sum(), sim.t)); rec["rng"].append(lid[act])
        rec["drive"].append(np.where(act)[0]); rec["step"].append(np.full(act.sum(), k))
        sim.step(expert_action(sim))
        if sim.done.all():
            break
    return {k: np.concatenate(v) for k, v in rec.items()}


def to_rays(rec, t_max):
    keep = rec["t"] <= t_max
    n = keep.sum()
    ox = np.repeat(rec["ox"][keep], N_RAYS)
    oy = np.repeat(rec["oy"][keep], N_RAYS)
    t = np.repeat(rec["t"][keep], N_RAYS)
```

```
    ang = np.tile(ANG, n)
    gt = rec["rng"][keep].ravel()
    return ox, oy, ang, t, gt


# ============================================================================ twin sim
class TwinSim(BatchSim):
    """Closed-loop simulation whose lidar comes from the NeRF instead of the simulator.
    The ped is shifted in time by +delta (field queried at t+delta); the true shifted
    pedestrian is still used for scoring (collision / gap)."""

    def __init__(self, field, P, rng, t_start, deltas, **kw):
        super().__init__(P, rng, t_start=t_start, **kw)
        self.field, self.deltas = field, np.asarray(deltas, float)

    def lidar(self):
        B = self.B
        ox = np.repeat(self.x + SENSOR_DX, N_RAYS)
        oy = np.repeat(self.ey, N_RAYS)
        t = np.repeat(np.clip(self.t + self.deltas, 0, self.field.t_max), N_RAYS)
        ang = np.tile(ANG, B)
        r = self.field.render(ox, oy, ang, t, use_dyn=getattr(self, "use_dyn", True)).reshape(B, N_RAYS)
        r = r + self.rng.standard_normal(r.shape) * self.noise[:, None]
        return np.clip(r, 0, R_MAX)
```

B.7 stack.py

```
"""Behaviour tree + perception + monitor + space-time lattice planner + smoothing + tracking + faults.
Pure NumPy, batched over scenarios (same simulator as the rest of the project)."""
import numpy as np
from sim import (BatchSim, DT, N_STEPS, EGO_L, EGO_W, A_MIN, A_MAX, COS, SIN, R_MAX, SENSOR_DX,
                 WALL_Y, PED_Y0, PED_Y1, PC_Y0, PC_Y1, rect_circle_gap)

# ------------------------------------------------------------------ vectorised behaviour tree
FAIL, SUCC, RUN = 0, 1, 2


class Cond:
    def __init__(s, name, fn): s.name, s.fn = name, fn
    def tick(s, c, m): return np.where(s.fn(c), SUCC, FAIL)


class Act:
    def __init__(s, name, fn, ident): s.name, s.fn, s.id = name, fn, ident
    def tick(s, c, m):
        c.a = np.where(m, s.fn(c), c.a); c.chosen = np.where(m, s.id, c.chosen)
        return np.full(len(m), RUN)


class Sel:
    def __init__(s, *k): s.k = k
    def tick(s, c, m):
        res, rem = np.full(len(m), FAIL), m.copy()
        for ch in s.k:
            st = ch.tick(c, rem); d = rem & (st != FAIL)
            res = np.where(d, st, res); rem &= ~d
        return res


class Seq:
    def __init__(s, *k): s.k = k
    def tick(s, c, m):
        res, rem = np.full(len(m), SUCC), m.copy()
        for ch in s.k:
            st = ch.tick(c, rem); f = rem & (st != SUCC)
            res = np.where(f, st, res); rem &= ~f
        return res


V_CRUISE, V_OCCL, V_BAD = 12.0, 8.0, 4.0
T_IN_MARGIN, T_OUT_MARGIN = 0.3, 0.8 # timing margins on the crossing prediction [s]
V_PED_WC = 3.5 # worst-case start speed of a pedestrian waiting at the curb [m/s]
```

```
46
47
48  def a_emerg(c): return -c.abrk
49  def a_brake(c):
50      b = -np.clip(1.3 * c.a_req + 0.3, 0.0, c.abrk)
51      return np.minimum(np.where(c.d <= 0, -c.abrk, b), a_cruise(c))
52  def a_cruise(c):
53      vd = np.where(c.bad, V_BAD, np.where(c.occl, V_OCCL, V_CRUISE))
54      vd = np.where(c.abrk < 8.0, np.minimum(vd, 6.0), vd)
55      return np.clip(vd - c.v, -3.0, 2.5)
56
57
58  def a_yield(c): return np.minimum(a_cruise(c), 0.0)
59
60
61  def build_tree():
62      """Priority: 1 emergency brake > 2 brake to stop line > 3 yield (never accelerate in conflict) > 4
            cruise (speed-capped near occluder / faults)."""
63      return Sel(Seq(Cond("emerg", lambda c: c.emerg), Act("EMERGENCY_BRAKE", a_emerg, 0)),
64                 Seq(Cond("brake", lambda c: c.brake_need), Act("BRAKE_TO_LINE", a_brake, 1)),
65                 Seq(Cond("conflict", lambda c: c.conflict), Act("YIELD_NO_ACCEL", a_yield, 3)),
66                 Act("CRUISE", a_cruise, 2))
67
68
69  class Ctx: pass
70
71
72  def derive(c, sim, est, bad, abrk):
73      """Fill the blackboard from ego state + pedestrian estimate (valid, y, vy, xp, pr)."""
74      valid, y, vy, xp, pr = est
75      x, v = sim.x, sim.v
76      xf, rear = x + EGO_L / 2, x - EGO_L / 2
77      band = EGO_W / 2 + pr + 0.4
78      xs, xc = xp - pr - 0.5, xp + pr + 0.5
79      in_band = np.abs(y) < band
80      vy = np.where((vy >= -0.05) & (~in_band) & (y > 0), -V_PED_WC, vy) # curb pedestrian may start at any
            time
81      moving = vy < -0.05
82      with np.errstate(divide="ignore", invalid="ignore"):
83          t_in = np.where(moving, np.maximum((y - band) / (-vy) - T_IN_MARGIN, 0.0), 0.0)
84          t_out = np.where(moving, np.maximum((y + band) / (-vy), 0.0), np.inf)
85          gone = np.where(np.isinf(t_out), False, (xf + v * (t_out + T_OUT_MARGIN)) < xs)
86      safe = ((~moving) & (~in_band)) | (moving & (y < -band)) | ((rear + v * t_in) > xc) | gone
87      c.conflict = valid & ~safe
88      c.d = xs - xf
89      c.a_req = v ** 2 / (2.0 * np.maximum(c.d, 0.2))
90      c.emerg = c.conflict & ((c.a_req > 0.75 * abrk) | (c.d <= 0))
91      c.brake_need = c.conflict & ((c.a_req > 0.8) | (c.d <= 0))
92      c.occl = (sim.pc_len > 0) & (sim.pc_x0 - xf < 35.0) & (xf < sim.pc_x1)
93      c.bad, c.abrk, c.v = bad, abrk, v
94
95
96  def tick(tree, c, B):
97      c.a, c.chosen = np.full(B, np.nan), np.full(B, -1)
98      tree.tick(c, np.ones(B, bool))
99      return c.a
100
101
102 def oracle_est(sim):
103     y, vy = sim.ped_state()
104     return np.ones(sim.B, bool), y, vy, sim.xp, sim.pr
105
106
107 # ------------------------------------------------------------------ perception from lidar
108 class Perception:
109     K = 10
110
111     def __init__(s, B):
112         s.hy, s.ht = np.full((B, s.K), np.nan), np.zeros((B, s.K))
113         s.last, s.x, s.y = np.full(B, -9.0), np.zeros(B), np.zeros(B)
114
115     def update(s, sim, lid, t):
116         ox, oy = (sim.x + SENSOR_DX)[:, None], sim.ey[:, None]
117         X, Y = ox + lid * COS, oy + lid * SIN
118         tw = np.where(SIN > 0, (WALL_Y - oy) / SIN, (-WALL_Y - oy) / SIN)
```

```
        inpc = ((X > sim.pc_x0[:, None] - 0.5) & (X < sim.pc_x1[:, None] + 0.5) & (Y > PC_Y0 - 0.5)
               & (Y < PC_Y1 + 0.5) & (sim.pc_len[:, None] > 0)) # static map filter
        m = (lid < tw - 1.0) & (lid < R_MAX - 0.5) & ~inpc
        n = m.sum(1)
        cx, cy = (X * m).sum(1) / np.maximum(n, 1), (Y * m).sum(1) / np.maximum(n, 1)
        det = (n >= 1) & (np.abs(cy) <= 6.5)
        s.hy = np.roll(s.hy, -1, 1); s.hy[:, -1] = np.where(det, cy, np.nan)
        s.ht = np.roll(s.ht, -1, 1); s.ht[:, -1] = t
        s.x, s.y, s.last = np.where(det, cx, s.x), np.where(det, cy, s.y), np.where(det, t, s.last)
        ok = ~np.isnan(s.hy); cnt = ok.sum(1)
        hy, ht = np.nan_to_num(s.hy), s.ht
        tm = (ok * ht).sum(1) / np.maximum(cnt, 1); ym = (ok * hy).sum(1) / np.maximum(cnt, 1)
        num = (ok * (ht - tm[:, None]) * (hy - ym[:, None])).sum(1)
        den = (ok * (ht - tm[:, None]) ** 2).sum(1)
        vy = np.where((cnt >= 4) & (den > 1e-6), num / np.maximum(den, 1e-9), np.nan)
        vy = np.where(np.isnan(vy), np.where(s.y > 0, -1.5, 0.0), vy) # pessimistic default
        valid = (t - s.last) <= 1.0
        y = np.clip(s.y + vy * (t - s.last), PED_Y1, PED_Y0)
        return valid, y, vy, s.x.copy(), np.full(len(vy), 0.6)


class Monitor:
    """Sensor-health (range sanity vs. known walls, frozen frame) and brake-health (achieved vs commanded).
       """

    def __init__(s, B):
        s.until, s.prev, s.pv, s.brk = np.zeros(B), None, None, np.zeros(B, bool)

    def update(s, sim, lid, t, a_prev):
        oy = sim.ey[:, None]
        tw = np.where(SIN > 0, (WALL_Y - oy) / SIN, (-WALL_Y - oy) / SIN)
        w = tw < 30
        frac = ((lid > tw + 1.5) & w).sum(1) / w.sum(1)
        frozen = np.zeros(sim.B, bool) if s.prev is None else np.all(lid == s.prev, 1)
        s.until = np.where((frac > 0.3) | frozen, t + 2.0, s.until)
        if s.pv is not None:
            ach = (s.pv - sim.v) / DT
            s.brk |= (a_prev < -3.5) & (ach < 0.7 * (-a_prev)) & (sim.v > 0.5)
        s.prev, s.pv = lid.copy(), sim.v.copy()
        return t < s.until, np.where(s.brk, 4.0, 8.0)


# ------------------------------------------------------------------- fault injection
FAULTS = ["dropout", "stuck", "bias", "latency", "brake"]


class FaultySim(BatchSim):
    """Fault window starts ‘pre‘ s before the pedestrian starts to cross (worst timing)."""

    def __init__(s, P, rng, fault=None, pre=1.0, dur=2.0, **kw):
        super().__init__(P, rng, **kw)
        s.fault, s.f0 = fault, s.t_start - pre
        s.f1, s.buf, s.fz = s.f0 + dur, [], None

    def lidar(s):
        r, f = super().lidar(), s.fault
        if f is None: return r
        act = ((s.t >= s.f0) & (s.t < s.f1))[:, None]
        if f == "dropout": return np.where(act, R_MAX, r)
        if f == "stuck":
            s.fz = r.copy() if s.fz is None else np.where(act, s.fz, r)
            return np.where(act, s.fz, r)
        if f == "bias": return np.where((s.t >= s.f0)[:, None], np.minimum(r * 1.25, R_MAX), r)
        if f == "latency":
            s.buf = (s.buf + [r])[-5:]
            return np.where((s.t >= s.f0)[:, None], s.buf[max(0, len(s.buf) - 4)], r)
        return r

    def step(s, a):
        if s.fault == "brake": a = np.where(s.t >= s.f0, np.maximum(a, -4.0), a)
        return super().step(a)


def run_bt(P, rng, fault=None, mode="perc", monitor=False, hist=False):
    B = len(P)
```

```
    sim = FaultySim(P, rng, fault)
    per, mon, tree, a_prev = Perception(B), Monitor(B), build_tree(), np.zeros(B)
    H = []
    for k in range(N_STEPS):
        lid, t = (None if mode == "oracle" else sim.lidar()), sim.t
        est = oracle_est(sim) if mode == "oracle" else per.update(sim, lid, t)
        bad, abrk = mon.update(sim, lid, t, a_prev) if monitor else (np.zeros(B, bool), np.full(B, 8.0))
        c = Ctx(); derive(c, sim, est, bad, abrk)
        a = tick(tree, c, B); a_prev = a
        if hist: H.append((sim.x.copy(), sim.v.copy(), c.chosen.copy()))
        sim.step(a)
        if sim.done.all(): break
    return sim, H


# ------------------------------------------------------------------ space-time lattice planner
ACTS, NSUB = np.array([-6.0, -3.0, -1.5, 0.0, 1.0, 2.0]), 5


def lattice_plan(x0, v0, ped, vd, H=6.0, dtp=0.5, margin=1.0):
    """Layered DP over (time, s, v) nodes; edges = constant-accel primitives checked against the
    predicted pedestrian occupancy in space-time. Returns accel profile at DT resolution or None."""
    tau = np.arange(1, NSUB + 1) * dtp / NSUB
    S, V, C = np.array([x0]), np.array([v0]), np.zeros(1)
    par, acc = [], []
    for k in range(int(H / dtp)):
        n = len(S)
        s0, vv, c0 = np.repeat(S, len(ACTS)), np.repeat(V, len(ACTS)), np.repeat(C, len(ACTS))
        a, pid = np.tile(ACTS, n), np.repeat(np.arange(n), len(ACTS))
        tstop = np.where(a < 0, vv / np.maximum(-a, 1e-9), np.inf)
        te = np.minimum(tau[None, :], tstop[:, None])
        pos = s0[:, None] + vv[:, None] * te + 0.5 * a[:, None] * te ** 2
        v1 = np.maximum(vv + a * dtp, 0.0)
        ok = np.ones(len(a), bool)
        if ped is not None:
            valid, y0, vy, xp, pr = ped
            py = np.clip(y0 + vy * (k * dtp + tau), PED_Y1, PED_Y0)[None, :]
            ok = (rect_circle_gap(pos, 0.0, xp, py, pr) > margin).all(1)
        cost = c0 + dtp * ((vd - 0.5 * (vv + v1)) ** 2 + 0.15 * a ** 2)
        if not ok.any(): return None
        idx = np.where(ok)[0]
        idx = idx[np.argsort(cost[idx])]
        key = np.round(pos[idx, -1] / 0.5).astype(int) * 100 + np.round(v1[idx] / 0.5).astype(int)
        idx = idx[np.unique(key, return_index=True)[1]]
        S, V, C = pos[idx, -1], v1[idx], cost[idx]
        par.append(pid[idx]); acc.append(a[idx])
    j, prof = int(np.argmin(C)), []
    for k in reversed(range(len(par))):
        prof.append(acc[k][j]); j = par[k][j]
    return np.repeat(prof[::-1], int(round(dtp / DT)))


def rollout(x0, v0, a):
    v, s = v0, x0
    S, V = [s], [v]
    for ak in a:
        v2 = max(v + ak * DT, 0.0)
        s += 0.5 * (v + v2) * DT; v = v2
        S.append(s); V.append(v)
    return np.array(S), np.array(V)


def smooth(s_lat, lam=2e-5, band=0.4):
    """Jerk-minimising refinement: min |s-s_lat|^2 + lam |s'''|^2, s0,s1 pinned, then clipped to a
    tube around the (collision-checked) lattice path."""
    N = len(s_lat)
    D = np.zeros((N - 3, N))
    for i in range(N - 3): D[i, i:i + 4] = np.array([-1, 3, -3, 1]) / DT ** 3
    W = np.eye(N); W[0, 0] = W[1, 1] = 1e4
    s = np.linalg.solve(W + lam * D.T @ D, W @ s_lat)
    return np.clip(s, s_lat - band, s_lat + band)


def run_planner(P, rng, use_smooth=True, kp=2.0, kd=2.5, mismatch=1.0, lag=0.25, supervisor=False):
    B = len(P)
```

```
    sim = BatchSim(P, rng)
    per, tree = Perception(B), build_tree()
    aact, plan, pk = np.zeros(B), [None] * B, np.full(B, -99)
    ex, A, terr, jerk_lat, jerk_sm, tpass = [], [], [], [], [], np.full(B, np.nan)
    for k in range(N_STEPS):
        lid, t = sim.lidar(), sim.t
        est = per.update(sim, lid, t)
        occl = (sim.pc_len > 0) & (sim.pc_x0 - (sim.x + EGO_L / 2) < 35.0) & ((sim.x + EGO_L / 2) < sim.
            pc_x1)
        acmd = np.full(B, -8.0)
        for i in range(B):
            if sim.done[i]: continue
            if k - pk[i] >= 5 or plan[i] is None:
                ped = tuple(float(e[i]) if np.ndim(e) else float(e) for e in est)
                al = lattice_plan(sim.x[i], sim.v[i], ped if ped[0] else None, 8.0 if occl[i] else 12.0)
                if al is None: plan[i] = None; pk[i] = k; continue
                sl, _ = rollout(sim.x[i], sim.v[i], al)
                ss = smooth(sl) if use_smooth else sl
                vr = np.gradient(ss, DT); ar = np.gradient(vr, DT)
                if use_smooth: jerk_sm.append(np.abs(np.diff(ar)).max() / DT); jerk_lat.append(np.abs(np.
                    diff(al)).max() / DT)
                plan[i], pk[i] = (ss, vr, ar), k
            if plan[i] is None: continue
            j = min(k - pk[i], len(plan[i][0]) - 1)
            sr, vr, ar = plan[i][0][j], plan[i][1][j], plan[i][2][j]
            acmd[i] = ar + kp * (sr - sim.x[i]) + kd * (vr - sim.v[i])
            terr.append(sr - sim.x[i])
        if supervisor: # simplex: BT safety layer can only reduce acceleration
            c = Ctx(); derive(c, sim, est, np.zeros(B, bool), np.full(B, 8.0))
            acmd = np.minimum(acmd, tick(tree, c, B))
        acmd = np.clip(acmd, A_MIN, A_MAX)
        aact += (acmd - aact) * DT / lag
        aeff = np.where(aact < 0, aact * mismatch, aact)
        A.append(np.where(sim.done, 0.0, aeff))
        was = sim.passed.copy()
        sim.step(aeff)
        tpass = np.where(np.isnan(tpass) & sim.passed & ~was, sim.t, tpass)
        if sim.done.all(): break
    A = np.array(A)
    return sim, dict(rms_track=float(np.sqrt(np.mean(np.square(terr)))) if terr else 0.0,
                     rms_acc=float(np.sqrt((A ** 2)[A != 0].mean())),
                     jerk95=float(np.percentile(np.abs(np.diff(A, axis=0)) / DT, 95)),
                     jerk_lat=float(np.mean(jerk_lat)) if jerk_lat else 0.0,
                     jerk_sm=float(np.mean(jerk_sm)) if jerk_sm else 0.0,
                     t_pass=float(np.nanmean(tpass)))
```

B.8 datasets.py

```
"""Convert real pedestrian tracks into scenario vectors z = (v0, dist, ped_speed, tau, occl, ped_r,
    dropout, noise).
Supported input: KITTI tracking label files (camera frame: x right, z forward, 10 Hz).
Usage: python datasets.py <label_file.txt> <ego_speed_m_s>"""
import sys
import numpy as np

FPS = 10.0


def load_kitti_tracks(path):
    tracks = {}
    for ln in open(path):
        f = ln.split()
        if len(f) < 17 or f[2] != "Pedestrian": continue
        frame, tid = int(f[0]), int(f[1])
        tracks.setdefault(tid, []).append((frame / FPS, float(f[13]), float(f[15]), float(f[11]))) # t, x,
            z, width
    return tracks


def to_scenarios(tracks, ego_speed, min_len=8):
    """Pedestrians whose lateral position changes sign (they cross the ego lane centre line)."""
    out = []
```

```
    for tid, rows in tracks.items():
        t, x, z, w = np.array(sorted(rows)).T
        if len(t) < min_len or x.min() * x.max() >= 0: continue
        k, b = np.polyfit(t, x, 1) # x(t) = k t + b
        tc = -b / k # time the pedestrian crosses the lane centre
        zc = float(np.interp(tc, t, z))
        out.append(dict(track=tid, v0=ego_speed, dist=zc, ped_speed=abs(k), tau=tc - zc / max(ego_speed, 0.1),
                        occl=0.0, ped_r=float(np.clip(np.median(w) / 2, 0.25, 1.0)), dropout=0.0, noise=0.05))
    return out


if __name__ == "__main__":
    for s in to_scenarios(load_kitti_tracks(sys.argv[1]), float(sys.argv[2])): print(s)
```

B.9 carla_bt.py

```
"""Run the project's behaviour tree in CARLA (>= 0.9.14, synchronous mode, ground-truth pedestrian state).
Needs a running CARLA server and 'pip install carla'. NOT executed in the project sandbox (no CARLA available).
CARLA frame: x forward, y right -> project frame: y_sim = -(y - y0).
Usage: python carla_bt.py --host localhost --port 2000"""
import argparse
import numpy as np
from sim import DT, EGO_L
from stack import build_tree, derive, tick, Ctx

PED_X, PED_Y0, PED_SPEED = 60.0, 5.5, 1.6


class View: # minimal stand-in for BatchSim fields used by derive()
    pc_len, pc_x0, pc_x1 = np.zeros(1), np.zeros(1), np.zeros(1)

    def __init__(s, x, v): s.x, s.v = np.array([x]), np.array([v])


def main(a):
    import carla
    cl = carla.Client(a.host, a.port); cl.set_timeout(20.0); w = cl.get_world()
    st = w.get_settings(); st.synchronous_mode, st.fixed_delta_seconds = True, DT; w.apply_settings(st)
    lib, o = w.get_blueprint_library(), w.get_map().get_spawn_points()[0].location
    ego = w.spawn_actor(lib.find("vehicle.tesla.model3"), carla.Transform(carla.Location(o.x, o.y, o.z + 0.5)))
    ped = w.spawn_actor(lib.filter("walker.pedestrian.*")[0], carla.Transform(carla.Location(o.x + PED_X, o.y - PED_Y0, o.z + 1.0)))
    hit = []; col = w.spawn_actor(lib.find("sensor.other.collision"), carla.Transform(), attach_to=ego); col.listen(lambda e: hit.append(e))
    tree, started = build_tree(), False
    try:
        for k in range(int(30 / DT)):
            v3, pl = ego.get_velocity(), ego.get_location(); pp = ped.get_location()
            x, v = pl.x - o.x, float(np.hypot(v3.x, v3.y))
            if x > PED_X - 25 and not started:
                ped.apply_control(carla.WalkerControl(carla.Vector3D(0, 1, 0), PED_SPEED)); started = True # +y_carla = -y_sim
            pv = ped.get_velocity()
            est = (np.ones(1, bool), np.array([-(pp.y - o.y)]), np.array([-pv.y]), np.array([pp.x - o.x]), np.array([0.4]))
            c = Ctx(); derive(c, View(x, v), est, np.zeros(1, bool), np.full(1, 8.0))
            acc = float(tick(tree, c, 1)[0])
            ego.apply_control(carla.VehicleControl(throttle=float(np.clip(acc / 3.0, 0, 1)), brake=float(np.clip(-acc / 8.0, 0, 1))))
            w.tick()
            if hit or x > PED_X + 8: break
        print("collision" if hit else "no collision", "| final x = %.1f m" % x)
    finally:
        col.destroy(); ped.destroy(); ego.destroy(); st.synchronous_mode = False; w.apply_settings(st)


if __name__ == "__main__":
```

```
    ap = argparse.ArgumentParser(); ap.add_argument("--host", default="localhost"); ap.add_argument("--port", type=int, default=2000)
    main(ap.parse_args())
```

B.10 embedded.py

```
"""Embedded deployment: export the learned policy to dependency-free C (float32 and int8 weights),
check bit-level agreement with NumPy, measure host latency/size, and re-run closed loop with int8 weights.
    """
import json, os, re, subprocess, ctypes, time
import numpy as np
from nn import MLP
from driving import OBS_DIM, evaluate, ACT_SCALE
from run_experiments import test_sets

from paths import RES, BUILD as _B
OUT, BUILD = str(RES), str(_B)


def arr(name, a, ctype="float"):
    return f"static const {ctype} {name}[{a.size}] = {{" + ",".join(f"{v:.9g}" if ctype == "float" else str(int(v)) for v in a.ravel()) + "};\n"


def gen_c(net, int8):
    src = "/* auto-generated by embedded.py */\n#include <stdint.h>\n#include <stddef.h>\n"
    sizes, scales = net.sizes, []
    for i, (W, b) in enumerate(zip(net.W, net.b)):
        if int8:
            s = float(np.abs(W).max() / 127.0); scales.append(s)
            src += arr(f"W{i}", np.round(W / s), "int8_t")
        else:
            scales.append(1.0); src += arr(f"W{i}", W)
        src += arr(f"B{i}", b)
    src += f"#define N0 {sizes[0]}\n#define N1 {sizes[1]}\n#define N2 {sizes[2]}\n#define N3 {sizes[3]}\n"
    src += "static void dense(const float*x,int n,int m,const %s*W,const float*b,float s,float*y,int relu){\n" % ("int8_t" if int8 else "float")
    src += " for(int j=0;j<m;j++){float a=0.f;for(int i=0;i<n;i++)a+=x[i]*(float)W[i*m+j];a=a*s+b[j];y[j]=(relu&&a<0.f)?0.f:a;}}\n"
    src += "float policy(const float*x){float h1[N1],h2[N2],o[N3];\n"
    src += f" dense(x,N0,N1,W0,B0,{scales[0]:.9e}f,h1,1);dense(h1,N1,N2,W1,B1,{scales[1]:.9e}f,h2,1);dense(h2,N2,N3,W2,B2,{scales[2]:.9e}f,o,0);\n"
    src += f" float a=o[0]*{ACT_SCALE}f;return a<-8.f?-8.f:(a>3.f?3.f:a);}}\n"
    src += "#ifdef BENCH\n#include <stdio.h>\n#include <time.h>\nint main(void){float x[N0];for(int i=0;i<N0;i++)x[i]=0.01f*i;volatile float sink=0;\n"
    src += " struct timespec t0,t1;clock_gettime(CLOCK_MONOTONIC,&t0);for(int k=0;k<200000;k++){x[0]=(float)(k&15)*0.01f;sink+=policy(x);}\n"
    src += " clock_gettime(CLOCK_MONOTONIC,&t1);printf(\"%.1f\\n\",((t1.tv_sec-t0.tv_sec)*1e9+(t1.tv_nsec-t0.tv_nsec))/200000.0);return 0;}\n#endif\n"
    return src


def numpy_int8(net):
    q = MLP(net.sizes, np.random.default_rng(0)); q.b = [b.copy() for b in net.b]
    q.W = [np.round(W / (np.abs(W).max() / 127.0)) * (np.abs(W).max() / 127.0) for W in net.W]
    return q


if __name__ == "__main__":
    net = MLP([OBS_DIM, 64, 64, 1], np.random.default_rng(0), out_scale=0.1); net.load(f"{OUT}/policy_A_seed0.npz")
    R = dict(params=int(sum(w.size + b.size for w, b in zip(net.W, net.b))), macs=int(sum(w.size for w in net.W)))
    X = np.random.default_rng(0).random((500, OBS_DIM)).astype(np.float32)
    ref = {"float32": np.clip(net.forward(X)[:, 0] * ACT_SCALE, -8, 3), "int8": np.clip(numpy_int8(net).forward(X)[:, 0] * ACT_SCALE, -8, 3)}
    for tag, i8 in (("float32", False), ("int8", True)):
        c = f"{BUILD}/policy_{tag}.c"; open(c, "w").write(gen_c(net, i8))
        for opt in ("-O2", "-Os"):
            exe = f"{BUILD}/bench_{tag}{opt}"
            subprocess.run(["gcc", opt, "-DBENCH", c, "-o", exe], check=True)
            ns = float(subprocess.run([exe], capture_output=True, text=True, check=True).stdout)
```

```
            subprocess.run(["gcc", opt, "-c", c, "-o", f"{exe}.o"], check=True)
            sz = subprocess.run(["size", f"{exe}.o"], capture_output=True, text=True).stdout.split("\n")[1].split()
            R[f"{tag}{opt}"] = dict(ns_per_inference=ns, text_bytes=int(sz[0]), data_bytes=int(sz[1]))
        subprocess.run(["gcc", "-O2", "-shared", "-fPIC", c, "-o", f"{BUILD}/lib_{tag}.so"], check=True)
        lib = ctypes.CDLL(f"{BUILD}/lib_{tag}.so"); lib.policy.restype = ctypes.c_float
        lib.policy.argtypes = [ctypes.POINTER(ctypes.c_float)]
        out = np.array([lib.policy(x.ctypes.data_as(ctypes.POINTER(ctypes.c_float))) for x in X])
        R[f"{tag}_max_abs_err_vs_numpy"] = float(np.abs(out - ref[tag]).max())
    Tn, Th, _ = test_sets(); q = numpy_int8(net)
    for nm, m in (("float32", net), ("int8", q)):
        R[f"closed_loop_{nm}"] = {k: float(evaluate(P, m, seed=1)["collided"].mean()) for k, P in (("normal", Tn), ("hazard", Th))}
    R["int8_vs_float_mean_abs_action_diff"] = float(np.abs(ref["int8"] - ref["float32"]).mean())
    json.dump(R, open(f"{OUT}/embedded.json", "w"), indent=1); print(json.dumps(R, indent=1))
```

B.11 run_experiments.py

```
"""Experiment 2: does diffusion-generated edge-case data make the driving policy safer?

Arms (equal total number of training scenarios = 2700):
  A normal only (baseline)
  B 1500 normal + 1200 guided-diffusion edge cases (failure-seeking generator)
  C 1500 normal + 1200 unguided diffusion samples (generator control)
  D 1500 normal + 1200 uniform random scenarios (naive domain randomisation)
Repeated for several seeds. Test sets are fixed and independent of every generator.
"""
import json, os, sys, time
import numpy as np
from multiprocessing import Pool

from sim import sample_normal, sample_uniform, open_loop_gap
from driving import train_policy, evaluate
from scenario_gen import ScenarioDiffusion

from paths import RES
OUT = str(RES)
N_NORMAL_BASE, N_GEN = 1500, 1200
GUIDANCE = 8.0
SEEDS = [0, 1, 2, 3, 4]


def test_sets():
    rng = np.random.default_rng(1000)
    normal = sample_normal(rng, 800)
    U = sample_uniform(np.random.default_rng(1001), 12000)
    haz = U[open_loop_gap(U) < 0.3][:800]
    hard = (haz[:, 6] > 0.4) | (haz[:, 5] > 0.6) # dark clothing / odd costume
    return normal, haz, hard


def summarise(r, mask=None):
    m = np.ones(len(r["collided"]), bool) if mask is None else mask
    return dict(coll=float(r["collided"][m].mean()), passed=float(r["passed"][m].mean()),
                gap=float(r["min_gap"][m].mean()), n=int(m.sum()))


def run_seed(seed):
    t0 = time.time()
    rng = np.random.default_rng(seed)
    Tn, Th, hard = test_sets()
    P_norm = sample_normal(rng, N_NORMAL_BASE + N_GEN)

    # ---- baseline policy (arm A)
    A = train_policy(P_norm, seed)
    # ---- failure surrogate: closed-loop min gap of A on uniform scenarios
    Pu = sample_uniform(rng, 3000)
    ru = evaluate(Pu, A, seed=seed)
    y = np.clip(ru["min_gap"], 0, 5) / 5.0
    # ---- diffusion prior + surrogate
    gen = ScenarioDiffusion(seed)
    gen.fit_prior(sample_normal(rng, 20000), iters=6000)
```

```
    gen.fit_surrogate(Pu, y, iters=5000)
    G_guided = gen.sample(N_GEN, GUIDANCE, seed=seed + 100)
    G_free = gen.sample(N_GEN, 0.0, seed=seed + 200)
    G_unif = sample_uniform(rng, N_GEN)

    sets = {"A": P_norm,
            "B": np.concatenate([P_norm[:N_NORMAL_BASE], G_guided]),
            "C": np.concatenate([P_norm[:N_NORMAL_BASE], G_free]),
            "D": np.concatenate([P_norm[:N_NORMAL_BASE], G_unif])}
    res = {}
    for arm, P in sets.items():
        net = A if arm == "A" else train_policy(P, seed + 7)
        rn, rh = evaluate(Tn, net, seed=seed), evaluate(Th, net, seed=seed)
        res[arm] = {"normal": summarise(rn), "hazard": summarise(rh),
                    "hard": summarise(rh, hard), "hazard_easy": summarise(rh, ~hard)}
        print(f"[seed {seed}] arm {arm}: normal coll {res[arm]['normal']['coll']:.3f} | "
              f"hazard coll {res[arm]['hazard']['coll']:.3f} | hard {res[arm]['hard']['coll']:.3f}", flush=
                  True)

    # ---- diagnostics for how 'hard' each generated set is (for the report)
    diag = {}
    for name, S in [("normal", P_norm[:N_GEN]), ("unguided", G_free), ("guided", G_guided), ("uniform",
         G_unif)]:
        ra, re = evaluate(S, A, seed=seed), evaluate(S, "expert", seed=seed)
        diag[name] = dict(A_coll=float(ra["collided"].mean()), expert_coll=float(re["collided"].mean()),
                          ol_hazard=float((open_loop_gap(S) < 0.3).mean()),
                          A_gap=float(ra["min_gap"].mean()))

    extra = {}
    if seed == SEEDS[0]: # guidance-strength sweep + arrays for figures
        sweep = []
        for lam in [0, 2, 4, 6, 8, 10, 12]:
            S = gen.sample(800, float(lam), seed=500 + lam)
            sweep.append(dict(lam=lam, A_coll=float(evaluate(S, A, seed=1)["collided"].mean()),
                              expert_coll=float(evaluate(S, "expert", seed=1)["collided"].mean()),
                              ol_hazard=float((open_loop_gap(S) < 0.3).mean())))
        extra["sweep"] = sweep
        coll_g = evaluate(G_guided, A, seed=seed)["collided"]
        coll_n = evaluate(P_norm[:N_GEN], A, seed=seed)["collided"]
        np.savez(f"{OUT}/fig_data_seed0.npz", normal=P_norm[:N_GEN], free=G_free, guided=G_guided,
                 uniform=G_unif, coll_guided=coll_g, coll_normal=coll_n,
                 prior_loss=np.array(gen.loss_curve), surr_loss=np.array(gen.surr_curve),
                 dagger_loss=np.array(A.curve))
        A.save(f"{OUT}/policy_A_seed0.npz")
    return dict(seed=seed, res=res, diag=diag, extra=extra, time=time.time() - t0)


if __name__ == "__main__":

    Tn, Th, hard = test_sets()
    ex = {"normal": summarise(evaluate(Tn, "expert")), "hazard": summarise(evaluate(Th, "expert")),
          "hard": summarise(evaluate(Th, "expert"), hard)}
    ol = float((open_loop_gap(Th) < 0.3).mean())
    print("expert reference:", ex, flush=True)
    with Pool(2) as pool:
        runs = pool.map(run_seed, SEEDS)
    json.dump(dict(expert=ex, runs=runs, n_hard=int(hard.sum()), seeds=SEEDS,
                   guidance=GUIDANCE, n_normal=N_NORMAL_BASE, n_gen=N_GEN),
              open(f"{OUT}/experiments.json", "w"), indent=1)
    print("done", [round(r["time"]) for r in runs])
```

B.12 run_twin.py

```
"""Experiment 1: NeRF-style digital twin of a logged drive + what-if replay.

1. Log 10 expert drives through one scene (parked car + one crossing pedestrian).
2. Fit the spacetime neural field to the lidar logs by differentiable volume rendering.
3. Novel-viewpoint check on two held-out drives (different lateral offsets / speeds).
4. What-if: re-render the scene with the pedestrian shifted by `delta` seconds and drive
   the learned policy *inside the twin*; compare with the same what-if in the real
   simulator (ground truth).
"""
```

```python
import json, os, sys, time
import numpy as np
from nn import MLP
from sim import BatchSim, ped_traj, PED_Y0, N_RAYS, ANG, R_MAX, SENSOR_DX, N_STEPS
from driving import policy_action, OBS_DIM
from nerf2d import NeuralField, run_logged_drives, to_rays, TwinSim

from paths import RES
OUT = str(RES)
T0, T_FIELD = 1.2, 12.0
ITERS = int(sys.argv[1]) if len(sys.argv) > 1 else 5000
BASE = dict(dist=60.0, s=1.6, tau=0.0, occl=0.8, pr=0.35, drop=0.0)


def scene(v0, noise, pr=None):
    v0 = np.atleast_1d(np.asarray(v0, float))
    n = len(v0)
    cols = [v0, np.full(n, BASE["dist"]), np.full(n, BASE["s"]), np.full(n, BASE["tau"]),
            np.full(n, BASE["occl"]), np.full(n, BASE["pr"] if pr is None else pr),
            np.full(n, BASE["drop"]), np.full(n, noise)]
    return np.stack(cols, 1)


def load_policy():
    net = MLP([OBS_DIM, 64, 64, 1], np.random.default_rng(0), out_scale=0.1)
    net.load(f"{OUT}/policy_A_seed0.npz")
    return net


if __name__ == "__main__":

    t0 = time.time()
    # ---- 1. logged drives
    K = 10
    v_log, ey_log = np.linspace(8.0, 13.0, K), np.linspace(-0.8, 0.8, K)
    rec = run_logged_drives(scene(v_log, 0.03), np.full(K, T0), ey_log, v_log, seed=1)
    rays = to_rays(rec, T_FIELD)
    print(f"logged {K} drives -> {len(rays[0])} lidar rays", flush=True)

    # ---- 2. fit the field
    field = NeuralField(BASE["dist"], T_FIELD, seed=0)
    t1 = time.time()
    field.fit(rays, iters=ITERS)
    fit_s = time.time() - t1
    print(f"fit time {fit_s:.0f}s", flush=True)

    # ---- 3. novel viewpoints (held-out drives, noise-free ground truth)
    v_te, ey_te = np.array([9.5, 11.5]), np.array([1.3, -1.3])
    rec_te = run_logged_drives(scene(v_te, 0.0), np.full(2, T0), ey_te, v_te, seed=2)
    ox, oy, ang, tt, gt = to_rays(rec_te, T_FIELD)
    pred = field.render(ox, oy, ang, tt)
    pred_static = field.render(ox, oy, ang, tt, use_dyn=False)
    # rays whose nearest hit is the pedestrian: same poses, pedestrian removed
    keep = rec_te["t"] <= T_FIELD
    s0 = BatchSim(scene(np.full(keep.sum(), 10.0), 0.0, pr=1e-3), np.random.default_rng(0))
    s0.x, s0.ey = rec_te["ox"][keep] - SENSOR_DX, rec_te["oy"][keep].copy()
    gt_np = s0.lidar().ravel()
    ped_rays = np.abs(gt - gt_np) > 0.3
    nv = dict(mae=float(np.abs(pred - gt).mean()), rmse=float(np.sqrt(((pred - gt) ** 2).mean())),
              within_half_m=float((np.abs(pred - gt) < 0.5).mean()),
              ped_rays=int(ped_rays.sum()), n_rays=int(len(gt)),
              mae_ped=float(np.abs(pred - gt)[ped_rays].mean()) if ped_rays.any() else None,
              within_half_m_ped=float((np.abs(pred - gt)[ped_rays] < 0.5).mean()) if ped_rays.any() else
                  None,
              mae_static_only_on_ped=float(np.abs(pred_static - gt)[ped_rays].mean()) if ped_rays.any() else
                   None,
              mae_static_only=float(np.abs(pred_static - gt).mean()))
    print("novel view:", nv, flush=True)
    # range images for the first held-out drive (rays x time)
    sel = (rec_te["drive"] == 0) & (rec_te["t"] <= T_FIELD)
    ns = sel.sum()
    gt_img = rec_te["rng"][sel] # (steps, rays)
    ox1, oy1, tt1 = np.repeat(rec_te["ox"][sel], N_RAYS), np.repeat(rec_te["oy"][sel], N_RAYS), np.repeat(
        rec_te["t"][sel], N_RAYS)
    pr_img = field.render(ox1, oy1, np.tile(ANG, ns), tt1).reshape(ns, N_RAYS)
```

```
    st_img = field.render(ox1, oy1, np.tile(ANG, ns), tt1, use_dyn=False).reshape(ns, N_RAYS)
    t_img = rec_te["t"][sel]

    # ---- 4. what-if replay inside the twin vs the real simulator
    policy = load_policy()
    deltas = np.linspace(-3.0, 3.0, 13)
    speeds = np.array([8.0, 10.0, 12.0, 14.0])
    D, V = np.meshgrid(deltas, speeds)
    D, V = D.ravel(), V.ravel()
    P = scene(V, 0.03)
    real = BatchSim(P, np.random.default_rng(5), t_start=T0 - D)
    twin = TwinSim(field, P, np.random.default_rng(5), T0 - D, D)
    nopd = TwinSim(field, P, np.random.default_rng(5), T0 - D, D)
    nopd.use_dyn = False
    traj = {k: [] for k in ("real", "twin")}
    for k in range(N_STEPS):
        for name, sim in (("real", real), ("twin", twin), ("nopd", nopd)):
            if sim.done.all():
                continue
            obs = sim.observe()
            sim.step(policy_action(policy, obs))
            if name in traj:
                traj[name].append(sim.x.copy())
        if real.done.all() and twin.done.all() and nopd.done.all():
            break
    def pad(L, n=N_STEPS):
        a = np.array(L)
        return np.vstack([a, np.repeat(a[-1:], n - len(a), 0)]) if len(a) < n else a
    wi = dict(
        n_cfg=int(len(D)),
        outcome_agreement=float((real.collided == twin.collided).mean()),
        collision_real=float(real.collided.mean()), collision_twin=float(twin.collided.mean()),
        gap_mae=float(np.abs(np.minimum(real.min_gap, 20) - np.minimum(twin.min_gap, 20)).mean()),
        gap_corr=float(np.corrcoef(np.minimum(real.min_gap, 20), np.minimum(twin.min_gap, 20))[0, 1]),
        xfinal_mae=float(np.abs(real.x - twin.x).mean()),
        pass_agreement=float((real.passed == twin.passed).mean()),
        collision_no_ped_twin=float(nopd.collided.mean()),
        mean_gap_no_ped_twin=float(np.minimum(nopd.min_gap, 20).mean()),
    )
    print("what-if:", wi, flush=True)

    # ---- BEV density maps for the figures
    xs, ys = np.arange(40, 80.01, 0.25), np.arange(-8, 8.01, 0.25)
    bev_t = [2.0, 3.5, 5.0]
    bev = {f"bev_{t}": field.bev(xs, ys, t) for t in bev_t}
    bev["bev_static"] = field.bev(xs, ys, 0.0, "static")
    np.savez(f"{OUT}/twin.npz", gt_img=gt_img, pred_img=pr_img, static_img=st_img, t_img=t_img,
             loss=np.array(field.loss_curve), D=D, V=V,
             gap_real=np.minimum(real.min_gap, 20), gap_twin=np.minimum(twin.min_gap, 20),
             gap_nopd=np.minimum(nopd.min_gap, 20),
             col_real=real.collided, col_twin=twin.collided, col_nopd=nopd.collided,
             traj_real=pad(traj["real"]), traj_twin=pad(traj["twin"]), xs=xs, ys=ys, **bev)
    json.dump(dict(novel_view=nv, what_if=wi, iters=ITERS, n_logged_rays=int(len(rays[0])),
                   n_drives=K, fit_seconds=fit_s), open(f"{OUT}/twin.json", "w"), indent=1)
    print(f"total {time.time() - t0:.0f}s")
```

B.13 run_stack.py

```
"""Experiments for the planning/safety stack: verification, planner, tracking, fault injection."""
import itertools, json, os, time
import numpy as np
from sim import A_MIN, A_MAX
from stack import (Ctx, build_tree, tick, run_bt, run_planner, FAULTS)
from run_experiments import test_sets
from driving import evaluate
from nn import MLP
from driving import OBS_DIM

from paths import RES
OUT = str(RES)


```

```
def verify_tree():
    """Exhaustive check of the behaviour tree over its abstract state space: hazard level
    (0 none, 1 conflict, 2 must brake, 3 emergency) x occluder x sensor-bad x braking capability (8/4),
    executing the real action code."""
    tree, rows = build_tree(), {}
    for lv, o, bad, ab in itertools.product(range(4), [0, 1], [0, 1], [8.0, 4.0]):
        c = Ctx()
        c.conflict, c.brake_need, c.emerg = (np.array([lv >= k]) for k in (1, 2, 3))
        c.occl, c.bad = np.array([bool(o)]), np.array([bool(bad)])
        c.abrk, c.v = np.array([ab]), np.array([10.0])
        c.d = np.array([{0: 200.0, 1: 200.0, 2: 40.0, 3: 4.0}[lv]])
        c.a_req = c.v ** 2 / (2 * c.d)
        rows[(lv, o, bad, ab)] = (float(tick(tree, c, 1)[0]), int(c.chosen[0]))
    names = ["P1 total, within actuator limits", "P2 emergency => full brake", "P3 must-brake => brake in
         [-abrk,-0.3]",
            "P4 conflict => never accelerate", "P5 no conflict => comfortable accel", "P6 monotone in
                 hazard level"]
    P = dict.fromkeys(names, 0)
    for (lv, o, bad, ab), (a, ch) in rows.items():
        P[names[0]] += ch >= 0 and np.isfinite(a) and A_MIN <= a <= A_MAX
        P[names[1]] += lv < 3 or a == -ab
        P[names[2]] += lv != 2 or (-ab <= a <= -0.3)
        P[names[3]] += lv == 0 or a <= 0.0
        P[names[4]] += lv > 0 or (-3.0 <= a <= 2.5)
        P[names[5]] += lv == 0 or a <= rows[(lv - 1, o, bad, ab)][0]
    return len(rows), P


def bmc_grid():
    g = itertools.product([6., 8., 10., 12., 14.], [35., 45., 55., 70.], [0.8, 1.4, 2.2, 3.5],
                          np.arange(-6, 6.01, 0.5), [0., .5, 1.], [0.25, 0.6, 1.0])
    P = np.array([[v, d, s, t, o, r, 0.0, 0.05] for v, d, s, t, o, r in g])
    sim, _ = run_bt(P, np.random.default_rng(0), mode="oracle")
    bad = np.where(sim.collided)[0]
    return len(P), len(bad), P[bad][:8].tolist()


def stat(sim):
    return dict(coll=float(sim.collided.mean()), passed=float(sim.passed.mean()),
                gap=float(np.minimum(sim.min_gap, 20).mean()))


if __name__ == "__main__":
    t0, R = time.time(), {}
    n_abs, props = verify_tree()
    R["tree"] = dict(n_states=n_abs, props=props)
    print("tree", n_abs, props, flush=True)
    n, nbad, ex = bmc_grid()
    R["bmc"] = dict(n=n, violations=nbad, examples=ex)
    print("bmc", n, nbad, ex[:3], flush=True)

    Tn, Th, hard = test_sets()
    A = MLP([OBS_DIM, 64, 64, 1], np.random.default_rng(0), out_scale=0.1); A.load(f"{OUT}/policy_A_seed0.
         npz")
    N = 100
    R["compare"] = {
        "Learned policy A": {k: float(v.mean()) for k, v in evaluate(Th[:N], A, seed=1).items() if k in ("
             collided", "passed")},
        "BT (lidar perception)": stat(run_bt(Th[:N], np.random.default_rng(1))[0]),
        "BT (oracle perception)": stat(run_bt(Th[:N], np.random.default_rng(1), mode="oracle")[0]),
    }
    for nm, kw in [("Lattice+opt+PD", {}), ("Lattice+opt+PD+BT supervisor", dict(supervisor=True))]:
        sim, st = run_planner(Th[:N], np.random.default_rng(1), **kw)
        R["compare"][nm] = {**stat(sim), **st}
        print(nm, R["compare"][nm], flush=True)
    sim, _ = run_bt(Th, np.random.default_rng(1)); R["bt_full_hazard"] = stat(sim)
    sim, _ = run_bt(Tn, np.random.default_rng(1)); R["bt_full_normal"] = stat(sim)
    print("BT full", R["bt_full_hazard"], R["bt_full_normal"], flush=True)

    R["tracking"] = {}
    for nm, kw in [("feed-forward only", dict(kp=0.0, kd=0.0)), ("PD + feed-forward", {})]:
        for mm in (1.0, 0.8):
            sim, st = run_planner(Th[:60], np.random.default_rng(2), mismatch=mm, supervisor=False, **kw)
            R["tracking"][f"{nm} | brake gain {mm}"] = {**stat(sim), **st}
            print(nm, mm, R["tracking"][f"{nm} | brake gain {mm}"], flush=True)
```

```
    sim, st = run_planner(Th[:60], np.random.default_rng(2), use_smooth=False)
    R["no_smooth"] = {**stat(sim), **st}

    R["faults"] = {}
    for f in [None] + FAULTS:
        R["faults"][str(f)] = {}
        for mon in (False, True):
            sim, _ = run_bt(Th[:400], np.random.default_rng(3), fault=f, monitor=mon)
            R["faults"][str(f)]["monitor" if mon else "plain"] = stat(sim)
        print(f, R["faults"][str(f)], flush=True)
    R["seconds"] = time.time() - t0
    json.dump(R, open(f"{OUT}/stack.json", "w"), indent=1)
    print("done", round(R["seconds"]))
```

B.14 make_figures.py

```
"""Builds every figure and the LaTeX number macros from results/*.json|npz."""
import json, os, sys
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, Circle

from sim import (BatchSim, ped_traj, ANG, R_MAX, SENSOR_DX, EGO_L, EGO_W, PC_Y0, PC_Y1, WALL_Y,
                 PED_Y0, PED_Y1, N_RAYS, PNAMES)
from driving import expert_action

from paths import RES, FIG
R, F = str(RES), str(FIG)

# validated categorical slots (light mode): blue, orange, aqua, yellow
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
INK, INK2, GRID, SURF = "#0b0b0b", "#52514e", "#e4e3df", "#fcfcfb"
ARMC = {"A": BLUE, "B": ORANGE, "C": AQUA, "D": YELLOW}
ARML = {"A": "A normal only", "B": "B + guided diffusion", "C": "C + unguided diffusion",
        "D": "D + uniform random"}

plt.rcParams.update({
    "font.size": 9, "axes.edgecolor": GRID, "axes.labelcolor": INK2, "xtick.color": INK2,
    "ytick.color": INK2, "text.color": INK, "axes.facecolor": SURF, "figure.facecolor": "white",
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color": GRID,
    "grid.linewidth": 0.6, "axes.axisbelow": True, "legend.frameon": False, "axes.titlesize": 10,
    "axes.titleweight": "normal", "axes.titlecolor": INK})


def save(fig, name):
    fig.savefig(f"{F}/{name}.pdf", bbox_inches="tight")
    fig.savefig(f"{F}/{name}.png", dpi=170, bbox_inches="tight")
    plt.close(fig)


# ---------------------------------------------------------------- simulator overview
def fig_sim_overview():
    P = np.array([[10.0, 60.0, 1.6, 0.0, 0.8, 0.35, 0.0, 0.03]])
    rng = np.random.default_rng(3)
    sim = BatchSim(P, rng, t_start=np.array([1.2]))
    snaps = {}
    for k in range(80):
        if k in (0, 22, 36):
            snaps[k] = (sim.x[0], sim.v[0], sim.t, sim.lidar()[0].copy(), sim.ped_state()[0][0])
        sim.step(expert_action(sim))
    fig, axes = plt.subplots(3, 1, figsize=(7.4, 6.4))
    for ax, (k, (x, v, t, lid, py)) in zip(axes, snaps.items()):
        ax.set_xlim(x - 8, 66); ax.set_ylim(-8, 8); ax.set_aspect("equal"); ax.grid(False)
        ax.axhline(WALL_Y, color=INK2, lw=2); ax.axhline(-WALL_Y, color=INK2, lw=2)
        ax.add_patch(Rectangle((60 - 3 - 4.8, PC_Y0), 4.8, PC_Y1 - PC_Y0, color="#9a9890"))
        ax.plot([60, 60], [PED_Y0, PED_Y1], ls=":", color=INK2, lw=1)
        ox, oy = x + SENSOR_DX, 0.0
        for a, r in zip(ANG[::2], lid[::2]):
            ax.plot([ox, ox + r * np.cos(a)], [oy, oy + r * np.sin(a)], color=BLUE, lw=0.4, alpha=0.55)
        ax.add_patch(Rectangle((x - EGO_L / 2, -EGO_W / 2), EGO_L, EGO_W, color=BLUE, zorder=3))
```

```
        ax.add_patch(Circle((60, py), 0.35, color=ORANGE, zorder=4))
        ax.set_title(f"t = {t:.1f} s speed = {v:.1f} m/s", loc="left", color=INK2, fontsize=9)
        ax.set_ylabel("y [m]")
    for ax in axes:
        ax.set_xlabel("x [m]")
    fig.tight_layout()
    save(fig, "sim_overview")


# ---------------------------------------------------------------------- experiment 2
def fig_experiments():
    J = json.load(open(f"{R}/experiments.json"))
    runs = J["runs"]
    arms = ["A", "B", "C", "D"]
    sets = [("normal", "Normal test set"), ("hazard", "Hazard test set"), ("hard", "Hard-perception subset")]
    stats = {a: {s: np.array([r["res"][a][s]["coll"] for r in runs]) * 100 for s, _ in sets} for a in arms}
    fig, ax = plt.subplots(figsize=(7.4, 3.3))
    w, gap = 0.19, 0.012
    for i, a in enumerate(arms):
        xs = np.arange(len(sets)) + (i - 1.5) * (w + gap)
        m = [stats[a][s].mean() for s, _ in sets]
        e = [stats[a][s].std(ddof=1) for s, _ in sets]
        ax.bar(xs, m, w, color=ARMC[a], label=ARML[a], yerr=e, capsize=2.5,
               error_kw=dict(ecolor=INK2, lw=1))
        for x_, m_, e_ in zip(xs, m, e):
            ax.text(x_, m_ + e_ + 0.6, f"{m_:.1f}", ha="center", fontsize=7.5, color=INK2)
    ax.set_xticks(range(len(sets))); ax.set_xticklabels([n for _, n in sets])
    ax.set_ylabel("collision rate [%]"); ax.set_ylim(0, 30); ax.grid(axis="x", visible=False)
    ax.legend(ncol=2, loc="upper left", fontsize=8)
    ax.set_title(f"Collision rate (mean +/- std over {len(runs)} seeds; equal training-set size)", loc="left")
    save(fig, "exp_collision_rates")

    # guidance sweep
    sw = runs[0]["extra"]["sweep"]
    fig, ax = plt.subplots(figsize=(5.2, 3.0))
    ax.plot([s["lam"] for s in sw], [100 * s["A_coll"] for s in sw], "-o", color=ORANGE, lw=2, ms=5,
            label="baseline policy A")
    ax.plot([s["lam"] for s in sw], [100 * s["expert_coll"] for s in sw], "-s", color=BLUE, lw=2, ms=5,
            label="privileged expert")
    ax.set_xlabel("guidance strength $\\lambda$"); ax.set_ylabel("collision rate on generated set [%]")
    ax.legend(loc="upper left"); ax.set_ylim(0, 60)
    ax.set_title("Guided sampling finds failures that are avoidable", loc="left")
    save(fig, "exp_guidance_sweep")

    # parameter marginals
    d = np.load(f"{R}/fig_data_seed0.npz")
    keys = [("tau", "arrival offset $\\tau$ [s]"), ("ped_speed", "pedestrian speed [m/s]"),
            ("ped_r", "pedestrian radius [m]"), ("v0", "ego speed $v_0$ [m/s]")]
    fig, axs = plt.subplots(1, 4, figsize=(8.2, 2.5))
    for ax, (k, lab) in zip(axs, keys):
        j = PNAMES.index(k)
        lo, hi = np.percentile(np.concatenate([d["normal"][:, j], d["guided"][:, j]]), [0.5, 99.5])
        bins = np.linspace(lo, hi, 26)
        for arr, c, ls, name in [(d["normal"], BLUE, "-", "everyday data"),
                                 (d["free"], AQUA, "--", "unguided"), (d["guided"], ORANGE, "-", "guided")]:
            ax.hist(arr[:, j], bins=bins, density=True, histtype="step", color=c, lw=1.8, ls=ls, label=name)
        ax.set_xlabel(lab, fontsize=8); ax.set_yticks([])
    axs[0].legend(fontsize=7, loc="upper left")
    fig.suptitle("Generated scenario parameters vs. everyday data (seed 0)", x=0.01, ha="left", fontsize=10)
    fig.tight_layout()
    save(fig, "exp_marginals")

    # scatter of tau vs ped speed
    fig, axs = plt.subplots(1, 2, figsize=(7.4, 3.0), sharey=True, sharex=True)
    for ax, (S, coll, ttl) in zip(axs, [(d["normal"], d["coll_normal"], "everyday scenarios"),
                                        (d["guided"], d["coll_guided"], "guided-diffusion scenarios")]):
        ok = ~coll.astype(bool)
        ax.scatter(S[ok, 3], S[ok, 2], s=5, color=BLUE, alpha=0.45, lw=0, label="A avoids collision")
        ax.scatter(S[~ok, 3], S[~ok, 2], s=7, color=ORANGE, alpha=0.9, lw=0, label="A collides")
        ax.set_title(f"{ttl} ({100 * coll.mean():.0f}% failures)", loc="left", fontsize=9)
        ax.set_xlabel("arrival offset $\\tau$ [s]")
```

```
    axs[0].set_ylabel("pedestrian speed [m/s]")
    h, l = axs[0].get_legend_handles_labels()
    fig.legend(h, l, loc="lower center", ncol=2, fontsize=8, markerscale=2.5, bbox_to_anchor=(0.5, -0.06))
    fig.tight_layout()
    save(fig, "exp_scatter")

    # training curves
    fig, axs = plt.subplots(1, 3, figsize=(8.2, 2.4))
    for ax, (k, ttl) in zip(axs, [("dagger_loss", "DAgger policy (MSE)"), ("prior_loss", "diffusion prior ($\\epsilon$-MSE)"),
                                  ("surr_loss", "failure surrogate (MSE)")]):
        y = d[k]
        ax.plot(np.arange(len(y)) * 100, y, color=BLUE, lw=1.6)
        ax.set_yscale("log"); ax.set_title(ttl, loc="left", fontsize=9); ax.set_xlabel("iteration")
    fig.tight_layout()
    save(fig, "exp_training_curves")

    # numbers for the report
    def ms(a, s, key="coll", scale=100):
        v = np.array([r["res"][a][s][key] for r in runs]) * scale
        return v.mean(), v.std(ddof=1)
    rows = []
    for a in arms:
        n, h, hd = ms(a, "normal"), ms(a, "hazard"), ms(a, "hard")
        g, pas = ms(a, "hazard", "gap", 1), ms(a, "normal", "passed")
        rows.append((a, n, h, hd, g, pas))
    diag = {k: np.mean([r["diag"][k][m] for r in runs]) for k in runs[0]["diag"] for m in ["A_coll"]}
    diag = {k: {m: float(np.mean([r["diag"][k][m] for r in runs])) for m in runs[0]["diag"][k]}
            for k in runs[0]["diag"]}
    return J, rows, diag


# ---------------------------------------------------------------------- experiment 1
def fig_twin():
    T = json.load(open(f"{R}/twin.json"))
    d = np.load(f"{R}/twin.npz")
    t, ang = d["t_img"], np.rad2deg(ANG)
    # range images
    fig, axs = plt.subplots(1, 4, figsize=(8.4, 2.9), sharey=True)
    ext = [t[0], t[-1], ang[0], ang[-1]]
    for ax, (img, ttl, cm, vmax) in zip(axs, [(d["gt_img"], "ground-truth lidar", "Blues_r", R_MAX),
                                              (d["pred_img"], "NeRF render", "Blues_r", R_MAX),
                                              (d["static_img"], "static field only", "Blues_r", R_MAX),
                                              (np.abs(d["pred_img"] - d["gt_img"]), "|error| [m]", "Oranges",
                                               5)]):
        im = ax.imshow(img.T, aspect="auto", origin="lower", extent=ext, cmap=cm, vmin=0, vmax=vmax)
        ax.set_title(ttl, loc="left", fontsize=9); ax.set_xlabel("time [s]"); ax.grid(False)
        fig.colorbar(im, ax=ax, fraction=0.05, pad=0.02)
    axs[0].set_ylabel("ray angle [deg]")
    fig.tight_layout()
    save(fig, "twin_range_images")

    # BEV density maps with ground-truth pedestrian
    xs, ys = d["xs"], d["ys"]
    panels = [("bev_static", "static field", None)] + [(f"bev_{tt}", f"static + dynamic, t = {tt} s", tt)
                                                       for tt in (2.0, 3.5, 5.0)]
    fig, axs = plt.subplots(2, 2, figsize=(7.6, 4.6), sharex=True, sharey=True)
    for ax, (key, ttl, tt) in zip(axs.ravel(), panels):
        ax.imshow(np.minimum(d[key], 25), origin="lower", cmap="Blues", vmin=0, vmax=25, aspect="equal",
                  extent=[xs[0] - 0.125, xs[-1] + 0.125, ys[0] - 0.125, ys[-1] + 0.125])
        ax.grid(False); ax.set_title(ttl, loc="left", fontsize=9)
        if tt is not None:
            py = ped_traj(np.array(1.6), np.array(1.2), tt)[0]
            ax.add_patch(Circle((60, float(py)), 0.9, fill=False, ec=ORANGE, lw=1.4))
    for ax in axs[1]:
        ax.set_xlabel("x [m]")
    for ax in axs[:, 0]:
        ax.set_ylabel("y [m]")
    fig.tight_layout()
    save(fig, "twin_bev_density")

    # loss
    fig, ax = plt.subplots(figsize=(4.8, 2.8))
    ax.plot(np.arange(len(d["loss"])) * 50, d["loss"], color=BLUE, lw=1.6)
    ax.set_yscale("log"); ax.set_xlabel("iteration"); ax.set_ylabel("Huber loss on range [m]")
```

```
    ax.set_title("Field training", loc="left")
    save(fig, "twin_loss")

    # what-if comparison
    D, V = d["D"], d["V"]
    deltas, speeds = np.unique(D), np.unique(V)
    shape = (len(speeds), len(deltas))
    gr, gt_ = d["gap_real"].reshape(shape), d["gap_twin"].reshape(shape)
    cr, ct = d["col_real"].reshape(shape), d["col_twin"].reshape(shape)
    gr, gt_ = np.maximum(gr, 0), np.maximum(gt_, 0)
    fig, axs = plt.subplots(1, 5, figsize=(9.6, 2.9), gridspec_kw=dict(width_ratios=[1, 1, 0.05, 0.32,
        0.9], wspace=0.3))
    axs[3].axis("off")
    ext = [deltas[0] - 0.25, deltas[-1] + 0.25, 0, len(speeds)]
    for ax, (g, c, ttl) in zip(axs[:2], [(gr, cr, "real simulator"), (gt_, ct, "NeRF digital twin")]):
        im = ax.imshow(g, origin="lower", aspect="auto", cmap="Blues", vmin=0, vmax=5, extent=ext)
        ax.grid(False)
        for i in range(shape[0]):
            for j in range(shape[1]):
                if c[i, j]:
                    ax.text(deltas[j], i + 0.5, "\u00d7", ha="center", va="center", color=ORANGE, fontsize
                        =13, fontweight="bold")
        ax.set_yticks(np.arange(len(speeds)) + 0.5); ax.set_yticklabels([f"{s:.0f}" for s in speeds])
        ax.set_xlabel("pedestrian shift $\\Delta$ [s]"); ax.set_title(ttl, loc="left", fontsize=9)
    axs[0].set_ylabel("ego speed $v_0$ [m/s]")
    fig.colorbar(im, cax=axs[2], label="min gap [m] (\u00d7 = collision)")
    ax = axs[4]
    ax.scatter(np.maximum(d["gap_real"], 0), np.maximum(d["gap_twin"], 0), s=14, color=BLUE, alpha=0.8, lw
        =0)
    ax.plot([0, 5], [0, 5], color=INK2, lw=1, ls="--")
    ax.set_xlim(-0.2, 5); ax.set_ylim(-0.2, 5)
    ax.set_xlabel("min gap, real [m]"); ax.set_ylabel("min gap, twin [m]")
    ax.set_title(f"r = {T['what_if']['gap_corr']:.2f}", loc="left", fontsize=9)
    save(fig, "twin_whatif")

    # trajectories
    fig, ax = plt.subplots(figsize=(5.6, 3.1))
    n = d["traj_real"].shape[0]
    tt = (np.arange(n) + 1) * 0.1
    for dlt, c in [(-2.0, YELLOW), (0.0, ORANGE), (2.0, BLUE)]:
        i = int(np.where((D == dlt) & (V == 12.0))[0][0])
        ax.plot(tt, d["traj_real"][:, i], color=c, lw=2.2, label=f"real, $\\Delta$={dlt:+.0f} s")
        ax.plot(tt, d["traj_twin"][:, i], color=INK, lw=1.0, ls="--")
    ax.plot([], [], color=INK, lw=1, ls="--", label="digital twin")
    ax.axhline(60 - 0.35 - 2.25, color=INK2, lw=0.8, ls=":")
    ax.set_xlabel("time [s]"); ax.set_ylabel("ego position x [m]"); ax.legend(fontsize=7.5, loc="lower
        right")
    ax.set_title("Same policy, real vs. twin ($v_0$ = 12 m/s)", loc="left")
    save(fig, "twin_trajectories")
    return T


def macros(J, rows, diag, T):
    L = []
    def mac(n, v):
        L.append(f"\\newcommand{{\\{n}}}{{{v}}}")
    names = {"A": "A", "B": "B", "C": "C", "D": "D"}
    for a, n, h, hd, g, pas in rows:
        mac(f"coll{a}normal", f"{n[0]:.1f} \\pm {n[1]:.1f}")
        mac(f"coll{a}hazard", f"{h[0]:.1f} \\pm {h[1]:.1f}")
        mac(f"coll{a}hard", f"{hd[0]:.1f} \\pm {hd[1]:.1f}")
        mac(f"gap{a}hazard", f"{g[0]:.2f} \\pm {g[1]:.2f}")
        mac(f"pass{a}normal", f"{pas[0]:.1f} \\pm {pas[1]:.1f}")
    ex = J["expert"]
    mac("expertnormal", f"{100 * ex['normal']['coll']:.1f}"); mac("experthazard", f"{100 * ex['hazard']['
        coll']:.1f}")
    mac("experthard", f"{100 * ex['hard']['coll']:.1f}"); mac("expertgap", f"{ex['hazard']['gap']:.2f}")
    for k, v in diag.items():
        mac(f"diag{k}Acoll", f"{100 * v['A_coll']:.1f}"); mac(f"diag{k}Excoll", f"{100 * v['expert_coll
            ']:.1f}")
        mac(f"diag{k}OL", f"{100 * v['ol_hazard']:.1f}")
    nv, wi = T["novel_view"], T["what_if"]
    mac("nvMAE", f"{nv['mae']:.2f}"); mac("nvRMSE", f"{nv['rmse']:.2f}"); mac("nvWithin", f"{100 * nv['
        within_half_m']:.1f}")
    mac("nvMAEstatic", f"{nv['mae_static_only']:.2f}")
```

```
    mac("nvPedRays", f"{nv['ped_rays']}"); mac("nvAllRays", f"{nv['n_rays']}")
    if nv["mae_ped"] is not None:
        mac("nvMAEped", f"{nv['mae_ped']:.2f}"); mac("nvWithinPed", f"{100 * nv['within_half_m_ped']:.1f}")
        mac("nvMAEstaticped", f"{nv['mae_static_only_on_ped']:.2f}")
    mac("wiAgree", f"{100 * wi['outcome_agreement']:.1f}"); mac("wiCollReal", f"{100 * wi['collision_real']:.1f}")
    mac("wiCollTwin", f"{100 * wi['collision_twin']:.1f}"); mac("wiGapMAE", f"{wi['gap_mae']:.2f}")
    mac("wiGapCorr", f"{wi['gap_corr']:.2f}"); mac("wiXMAE", f"{wi['xfinal_mae']:.2f}")
    mac("wiCollNoPed", f"{100 * wi['collision_no_ped_twin']:.1f}"); mac("wiNcfg", f"{wi['n_cfg']}")
    mac("nerfRays", f"{T['n_logged_rays']}"); mac("nerfIters", f"{T['iters']}"); mac("nerfFit", f"{T['fit_seconds']:.0f}")
    mac("nSeeds", f"{len(J['seeds'])}"); mac("nNormal", f"{J['n_normal']}"); mac("nGen", f"{J['n_gen']}")
    mac("guidanceLam", f"{J['guidance']:.0f}"); mac("nHard", f"{J['n_hard']}")
    open(f"{R}/macros.tex", "w").write("\n".join(L) + "\n")


if __name__ == "__main__":
    fig_sim_overview()
    J, rows, diag = fig_experiments()
    print("experiment figures done")
    if os.path.exists(f"{R}/twin.json"):
        T = fig_twin()
        macros(J, rows, diag, T)
        print("twin figures + macros done")
```

B.15 make_figures_stack.py

```
"""Figures + LaTeX macros for the planning/safety stack (reads results/stack.json, embedded.json)."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import make_figures as mf # shared style + colours
from stack import lattice_plan, rollout, smooth
from sim import DT

from paths import RES, FIG
R, F = str(RES), str(FIG)
S, E = json.load(open(f"{R}/stack.json")), json.load(open(f"{R}/embedded.json"))
BLUE, ORANGE, AQUA, YELLOW, INK2 = mf.BLUE, mf.ORANGE, mf.AQUA, mf.YELLOW, mf.INK2

# ---- 1. controller comparison
names = list(S["compare"]); cols = [BLUE, ORANGE, AQUA, YELLOW, "#6250d6"]
vals = [100 * S["compare"][n].get("coll", S["compare"][n].get("collided")) for n in names]
fig, ax = plt.subplots(figsize=(7.4, 3.0))
ax.bar(range(len(names)), vals, color=cols[:len(names)], width=0.62)
for i, v in enumerate(vals): ax.text(i, v + 0.5, f"{v:.1f}", ha="center", fontsize=8, color=INK2)
ax.set_xticks(range(len(names))); ax.set_xticklabels([n.replace(" (", "\n(").replace("+PD+", "+PD\n+") for n in names], fontsize=7.5)
ax.set_ylabel("collision rate [%]"); ax.grid(axis="x", visible=False)
ax.set_title("Controllers on the same 100 hazard scenarios", loc="left")
mf.save(fig, "stack_compare")

# ---- 2. fault injection
fs = list(S["faults"]); lab = {"None": "nominal", "dropout": "lidar\ndropout", "stuck": "frozen\nframe", "bias": "range\nbias", "latency": "latency\n0.3 s", "brake": "brake\nfade"}
x = np.arange(len(fs)); fig, ax = plt.subplots(figsize=(7.4, 3.0))
for j, (k, c, n) in enumerate([("plain", BLUE, "BT, no monitor"), ("monitor", ORANGE, "BT + health monitor")]):
    v = [100 * S["faults"][f][k]["coll"] for f in fs]
    ax.bar(x + (j - 0.5) * 0.36, v, 0.34, color=c, label=n)
    for xi, vi in zip(x + (j - 0.5) * 0.36, v): ax.text(xi, vi + 0.4, f"{vi:.1f}", ha="center", fontsize=7.5, color=INK2)
ax.set_xticks(x); ax.set_xticklabels([lab[f] for f in fs], fontsize=8); ax.set_ylabel("collision rate [%]")
ax.set_ylim(0, 11); ax.legend(loc="upper center", fontsize=8, ncol=2); ax.grid(axis="x", visible=False)
ax.set_title("Fault injection (400 hazard scenarios)", loc="left")
mf.save(fig, "stack_faults")

# ---- 3. lattice vs optimised trajectory
ped = (True, 3.0, -1.6, 60.0, 0.6); al = lattice_plan(20.0, 10.0, ped, 8.0)
```

```
sl, vl = rollout(20.0, 10.0, al); ss = smooth(sl); vs = np.gradient(ss, DT); as_ = np.gradient(vs, DT)
t = np.arange(len(sl)) * DT
fig, axs = plt.subplots(1, 3, figsize=(8.6, 2.6))
axs[0].plot(t, sl, color=BLUE, lw=2, label="lattice"); axs[0].plot(t, ss, color=ORANGE, lw=1.4, ls="--", label="optimised")
axs[0].set_ylabel("position s [m]"); axs[0].legend(fontsize=7.5)
axs[1].plot(t[:-1], al if len(al) == len(t) - 1 else al, color=BLUE, lw=2); axs[1].plot(t, as_, color=ORANGE, lw=1.4)
axs[1].set_ylabel("acceleration [m/s$^2$]")
axs[2].plot(t[:-2], np.abs(np.diff(np.append(al, al[-1])))[:len(t) - 2] / DT, color=BLUE, lw=2)
axs[2].plot(t[:-1], np.abs(np.diff(as_)) / DT, color=ORANGE, lw=1.4); axs[2].set_ylabel("|jerk| [m/s$^3$]")
for a in axs: a.set_xlabel("time [s]")
fig.tight_layout(); mf.save(fig, "stack_trajectory")

# ---- macros
M = []
def mac(n, v): M.append(f"\\newcommand{{\\{n}}}{{{v}}}")
P = lambda x: f"{100 * x:.1f}"
mac("treeStates", S["tree"]["n_states"]); mac("treeProps", len(S["tree"]["props"])); mac("bmcN", S["bmc"]["n"]); mac("bmcViol", S["bmc"]["violations"])
for k, key in [("stackA", "Learned policy A"), ("stackBT", "BT (lidar perception)"), ("stackBTo", "BT (oracle perception)"),
               ("stackPl", "Lattice+opt+PD"), ("stackPs", "Lattice+opt+PD+BT supervisor")]:
    d = S["compare"][key]; mac(k + "Coll", P(d.get("coll", d.get("collided")))); mac(k + "Pass", P(d["passed"]))
    if "gap" in d: mac(k + "Gap", f"{d['gap']:.2f}")
d = S["compare"]["Lattice+opt+PD+BT supervisor"]
for k in ("rms_track", "rms_acc", "jerk95", "jerk_lat", "jerk_sm", "t_pass"): mac("pl" + k.replace("_", "").replace("95", "NF"), f"{d[k]:.2f}")
mac("btFullHz", P(S["bt_full_hazard"]["coll"])); mac("btFullNorm", P(S["bt_full_normal"]["coll"]))
for k, key in [("trFF", "feed-forward only | brake gain 1.0"), ("trFFm", "feed-forward only | brake gain 0.8"), ("trPD", "PD + feed-forward | brake gain 1.0"), ("trPDm", "PD + feed-forward | brake gain 0.8")]:
    mac(k + "Rms", f"{S['tracking'][key]['rms_track']:.3f}"); mac(k + "Coll", P(S["tracking"][key]["coll"])); mac(k + "Jerk", f"{S['tracking'][key]['jerk95']:.1f}")
mac("nsJerk", f"{S['no_smooth']['jerk95']:.1f}"); mac("nsColl", P(S["no_smooth"]["coll"]))
for f in S["faults"]:
    for k in ("plain", "monitor"): mac(f"f{f}{k}", P(S["faults"][f][k]["coll"]))
NM = {"float32": "Fl", "int8": "Iq"}
for t in ("float32", "int8"):
    n_ = NM[t]
    mac(f"emb{n_}ns", f"{E[t + '-O2']['ns_per_inference'] / 1000:.1f}"); mac(f"emb{n_}flash", f"{E[t + '-Os']['text_bytes'] / 1024:.1f}")
    mac(f"emb{n_}err", f"{E[t + '_max_abs_err_vs_numpy']:.1e}"); mac(f"emb{n_}n", P(E['closed_loop_' + t]['normal'])); mac(f"emb{n_}h", P(E['closed_loop_' + t]['hazard']))
mac("embParams", E["params"]); mac("embMacs", E["macs"])
open(f"{R}/macros_stack.tex", "w").write("\n".join(M) + "\n"); print("ok")
```

B.16 tests.py

```
"""Smoke tests: python tests.py"""
import os, tempfile
import numpy as np
from run_stack import verify_tree
from datasets import load_kitti_tracks, to_scenarios
from sim import sample_normal, open_loop_gap
from stack import run_bt


def test_tree():
    n, props = verify_tree()
    assert n == 32 and all(v == n for v in props.values()), props


def test_oracle_bt_safe_on_everyday():
    P = sample_normal(np.random.default_rng(0), 300)
    sim, _ = run_bt(P, np.random.default_rng(0), mode="oracle")
    assert sim.collided.mean() == 0.0


```

```
def test_kitti_adapter(): # synthetic KITTI-format track: pedestrian walking at 1.5 m/s across the lane centre
    rows = [f"{i} 7 Pedestrian 0 0 0 0 0 0 0 1.7 0.6 0.8 {-3.0 + 1.5 * i / 10:.3f} 0 40.0 0" for i in range(40)]
    with tempfile.NamedTemporaryFile("w", suffix=".txt", delete=False) as f: f.write("\n".join(rows))
    s = to_scenarios(load_kitti_tracks(f.name), ego_speed=10.0); os.unlink(f.name)
    assert len(s) == 1 and abs(s[0]["ped_speed"] - 1.5) < 1e-3 and abs(s[0]["dist"] - 40.0) < 1e-6 and abs(s[0]["tau"] - (2.0 - 4.0)) < 0.2, s


if __name__ == "__main__":
    for name, fn in list(globals().items()):
        if name.startswith("test_"): fn(); print("ok", name)
```

B.17 Makefile

```
# Full pipeline (about 15 min on 2 CPU cores). Run from the project root.
PY ?= python3
.PHONY: all test results figures pdf clean
all: results figures pdf
test:
	cd code && $(PY) tests.py
results:
	cd code && $(PY) run_experiments.py > ../logs/experiments.log
	cd code && $(PY) run_twin.py 5000 > ../logs/twin.log
	cd code && $(PY) run_stack.py > ../logs/stack.log
	cd code && $(PY) embedded.py > ../logs/embedded.log
figures:
	cd code && $(PY) make_figures.py && $(PY) make_figures_stack.py
pdf:
	latexmk -pdf -interaction=nonstopmode main.tex
clean:
	rm -rf build/* && latexmk -C main.tex
```